TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PULCHOWK CAMPUS

A

PROJECT REPORT

ON

A RETRIEVAL-BASED DOCUMENT Q&A SYSTEM WITH MCQ GENERATION USING A RAG ARCHITECTURE

SUBMITTED BY:

SACHITA ARYAL (PUL079BCT073)

SUJAL PANDIT (PUL079BCT084)

SUJAN BHATTARAI (PUL079BCT086)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

MARCH, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"A RETRIEVAL-BASED DOCUMENT Q&A SYSTEM WITH MCQ GENERATION USING A RAG ARCHITECTURE"** submitted by **Sachita Aryal**, **Sujal Pandit**, **Sujan Bhattarai** in partial fulfillment of the requirements for the Bachelor's degree in Electronics & Computer Engineering.

.....
Supervisor
Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.

.....
External examiner
Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.

Date of approval: _____

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.
Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Pulchowk Campus, Institute of Engineering, TU
Lalitpur, Nepal.

Acknowledgments

We would like to express our sincere gratitude to everyone who contributed to the successful completion of our project.

We are deeply thankful to the **Department of Electronics and Computer Engineering** for providing us with the necessary resources and a supportive environment to carry out this work effectively.

We extend our heartfelt appreciation to the **Head of the Department**, **Deputy Head**, and **IC Chair** for their invaluable guidance, constructive feedback, and continuous encouragement throughout the project. We are also grateful to all the faculty members and staff for their support, knowledge sharing, and motivation.

We would like to thank our seniors and peers for their guidance, helpful suggestions, and for sharing their experiences, which helped us overcome various challenges during the project.

We also acknowledge the support and cooperation of our friends, whose encouragement and collaborative discussions contributed significantly to the successful completion of this work.

Finally, we express our deepest gratitude to our family members for their constant encouragement, understanding, and unwavering moral support, which played a vital role in achieving this milestone.

Authors:
Sachita Aryal (079BCT073)
Sujal Pandit (079bct084)
Sujan Bhattarai(079bct086)

Abstract

Managing large collections of PDF documents is a persistent challenge for students, researchers, and professionals. Traditional keyword search tools fail to capture semantic meaning, while general-purpose AI assistants cannot access user-specific documents and raise privacy concerns when files are uploaded to external servers. This project presents DocuMind, a privacy-preserving desktop application that enables natural language question answering and automated multiple-choice question (MCQ) generation directly from user-provided PDF documents. The system implements a Retrieval-Augmented Generation (RAG) architecture combining windowed late chunking with the nomic-embed-text-v1.5 embedding model, ChromaDB with HNSW indexing for vector storage, and Groq's llama-3.1-8b-instant for answer generation. All document processing and retrieval operations run locally on CPU, with only the final generation step using a cloud API, ensuring that complete documents are never transmitted externally. The system was evaluated on a corpus of 31 documents totalling 904 pages and approximately one million tokens, using 171 manually curated ground-truth queries. The retrieval pipeline achieved a Recall@5 of 71.3% and a Mean Reciprocal Rank of 0.547, with a mean retrieval latency of 0.145 seconds on CPU-only hardware. The MCQ generation pipeline achieved 100% answer extractability and 88.9% format validity across all evaluation queries, with zero API errors. Indexing cost scales linearly with document length, and cached documents reload in under one second regardless of size. The results demonstrate that a local-first RAG system can deliver competitive retrieval performance and robust educational content generation on standard hardware, without compromising user privacy.

Contents

Page of Approval **ii**

Copyright **iii**

Acknowledgements **vi**

Abstract **vi**

Contents **viii**

List of Figures **ix**

List of Abbreviations **x**

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope 2

2 Literature Review **3**

2.1 Related Work 3

2.2 Related Theory 4

3 Methodology **6**

3.1 Methodology 6

3.1.1 Retrieval-Augmented Generation (RAG) Architecture 6

3.1.2 Document Processing and Chunking 6

3.1.3 Embedding Generation 7

3.1.4 Retrieval Algorithm 7

3.1.5 Answer Generation Workflow 8

3.1.6 MCQ Generation Process 8

3.2 Feasibility Study 9
3.3 Requirements Analysis 10
3.3.1 Functional Requirements 10
3.3.2 Non-Functional Requirements 10
3.3.3 Operational Requirements 11

4 Experimental Setup **12**
4.1 Evaluation Framework 12
4.2 Performance Benchmarking 13
4.3 Experimental Procedure 14
4.4 Validation Methods 14

5 System Design **15**
5.1 System Architecture 15
5.2 Document Processing Module 16
5.3 Embedding and Vector Storage Module 16
5.3.1 Embedding Model 16
5.3.2 Vector Database and Indexing 17
5.3.3 Query Processing and Retrieval Module 17
5.4 LLM Generation Module 17
5.4.1 LLM and Prompting 17
5.5 Class Diagram 18
5.6 Component Diagram 19
5.7 Use Case Diagram 19
5.8 Sequence Diagram 20

6 Result And Discussion **21**
6.1 System Interface 21
6.1.1 Question Answering Mode 21
6.1.2 Multiple Choice Questions Generation Mode 22
6.2 Tesing and Evaluation 22
6.2.1 Indexing Performance 22
6.2.2 Retrieval Performance 24
6.2.3 Retrieval Latency Distribution 28
6.2.4 MCQ Generation Quality 29
6.3 Discussion 30

7 Conclusion **31**

8 Limitation And Future Enhancements **32**

8.1 Limitation 32

8.2 Future Enhancements 33

References **34**

Appendix A Complete Evaluation Corpus **36**

Appendix B Per-Document Retrieval Results **38**

Appendix C Sample MCQ Generation Outputs **40**

Appendix D Per-Document Indexing Time Breakdown **42**

Appendix E Windowed Late Chunking: Algorithm and Implementation **44**

List of Figures

2.1 HNSW Visualization. Reproduced from Malkov et al.[1] 4
2.2 Chunking with chunk-size of 6 words and chunk-overlay of 2 words 5

5.1 System Architecture 15
5.2 Class Diagram 18
5.3 Component Diagram 19
5.4 Use Case Diagram 19
5.5 Sequence Diagram for query_mcq() 20

6.1 GUI Output in Question Answering Mode 21
6.2 GUI Output in MCQ Mode 22
6.3 Token count vs embedding time Linear fit $R^2 > 0.95$ 23
6.4 Average Indexing time breakdown per tier. Embedding dominates at 94% of total time 23
6.5 Recall@5 per document sorted best to worst. Dashed line is the overall mean (71.3%) 25
6.6 Retrieval outcome breakdown per document showing correct-page, off-by-one, and genuine miss counts 26
6.7 Recall reordered by page count 27
6.8 Per-query retrieval time distribution (n=171) 28
6.9 MCQ automatic metric pass rates 29

List of Abbreviations

AI	Artificial Intelligence
RAG	Retrieval-Augmented Generation
LLM	Large Language Models
MCQ	Multiple-Choice Question
BERT	Bidirectional Encoder Representations from Transformers
HNSW	Hierarchical Navigable Small World
BM25	Best Matching 25

1. Introduction

This chapter presents an overview of the minor project titled **"A Retrieval-Based Document Q&A System with MCQ Generation Using a RAG Architecture"**. It describes the challenges of managing large PDF document collections and the limitations of traditional search tools. The project aims to enable secure and efficient natural language queries, question answering, and MCQ generation from user documents.

1.1 Background

In today's digital age, students and researchers today work with many PDF documents like lecture notes, research papers, and textbooks. Finding specific information in these files takes a lot of time. Manual reading is slow, and simple keyword searches often miss important content.

AI chatbots like ChatGPT can answer questions, but they have problems. They cannot read your specific documents, only use their general training data. They sometimes give wrong information (hallucination) and require uploading files to external servers, which creates privacy risks for sensitive documents.

The proposed system **"A Retrieval-Based Document Q&A System with MCQ Generation Using a RAG Architecture"** solves these problems by creating a desktop application where users can ask questions about their documents in plain English. The system reads the documents, understands their content, and provides accurate answers. Importantly, every answer shows the exact page number and text from the source document, so users can verify the information.

The system also generates multiple-choice questions from documents automatically. Students can create practice tests, and teachers can make assessments quickly. Both features work using the same technology to ensure accuracy.

All document processing happens on the user's computer, keeping data private and secure. Only the final answer generation uses an internet connection, but no complete documents are sent online. This design works on regular computers without needing expensive hardware.

1.2 Problem Statement

Students, researchers, and professionals often manage large collections of digital documents, predominantly in PDF format. Manual search is time-consuming, and traditional AI tools have significant limitations. The key problems addressed by this project are the following:

1. **Context Limitation:** Standard LLMs cannot answer questions about a specific PDF (e.g.

"What is the budget in this uploaded invoice?") because the content of user-uploaded documents is not part of their training data.

2. **Inefficient Information Retrieval:** Traditional search methods (e.g., Ctrl+F) are keyword-based and cannot understand semantic context or intent. For example, searching for "cost" may miss a section labeled "expense."

3. **Privacy Concerns:** Many AI document tools require the upload of files to external servers, which is problematic for confidential or proprietary information such as legal, medical or business documents.

4. **Lack of Source Citations:** AI-generated answers often do not provide references or page numbers, making it difficult to verify accuracy or trace the original content.

5. **Limited Assessment Tools:** Educators and students lack reliable automated tools to generate accurate multiple-choice questions from document content for self-assessment or quizzes.

1.3 Objectives

1. Develop a PyQt5 Desktop App: To provide a user-friendly interface for document interaction.

2. Automate Processing: Extract text, perform intelligent chunking, and store semantic embeddings in ChromaDB.

3. Implement RAG: Retrieve relevant document sections to generate grounded, cited answers.

4. MCQ Generation: Automatically create practice questions and distractors from the content.

1.4 Scope

The scope of this project is limited to the design and development of a prototype desktop application for intelligent document querying and analysis. The system will support PDF document upload, semantic search using RAG-based retrieval, natural language question answering, and multiple-choice question generation. All outputs are intended to assist users in retrieving and understanding information efficiently, with full transparency and citation from the source documents.

2. Literature Review

This chapter presents a review of existing research and theoretical foundations relevant to the development of an intelligent document query and MCQ generation system. We examine prior work in retrieval-augmented generation, document processing, embedding techniques, and dense retrieval algorithms, followed by the theoretical concepts underlying these methods.

2.1 Related Work

Recent advancements in Retrieval-Augmented Generation (RAG) have significantly improved the performance of large language models (LLMs) on knowledge-intensive tasks by combining parametric knowledge with external document retrieval [2]. Lewis et al. [2] introduced a pioneering RAG framework that retrieves relevant passages from a large corpus and conditions the generation process on this retrieved context, effectively reducing hallucinations and improving factual accuracy.

Dense Passage Retrieval (DPR) techniques further enhance semantic search capabilities by encoding queries and documents into dense vector spaces, enabling retrieval based on semantic similarity rather than keyword matching [3]. Karpukhin et al. [3] demonstrated DPR's superiority over traditional sparse retrieval methods like BM25, showing improved retrieval accuracy in open-domain question answering.

A fundamental limitation of standard dense retrieval pipelines is that text chunks are embedded independently, discarding document-level context. Pronouns, acronyms, and implicit references that span chunk boundaries cannot be resolved within an isolated chunk embedding. Günther et al. [4] addressed this with late chunking, which runs the full document through the transformer encoder before performing chunk-level mean pooling, preserving coreference and contextual meaning within each chunk. Merola and Singh [5] further evaluated late chunking against contextual retrieval, finding that late chunking offers higher efficiency while contextual retrieval better preserves semantic coherence at greater computational cost. Our system extends the late chunking approach with a sliding window mechanism to handle documents that exceed the model's context limit, making it practical for large real-world document collections.

ChromaDB combined with Hierarchical Navigable Small World (HNSW) indexing offers efficient approximate nearest neighbor search, balancing speed and recall in vector-based retrieval systems [1, 6]. This combination has been leveraged in many recent RAG systems to enable scalable and low-latency semantic search on document embeddings [7].

Educational applications of RAG frameworks, such as automated MCQ generation from doc-

ument content, have begun to emerge, enabling personalized and scalable learning tools [7, 6]. These systems utilize retrieved context to formulate relevant and accurate assessment questions, improving over heuristic or template-based MCQ generation methods.

Despite these advances, existing systems often face challenges with privacy (due to cloud processing), lack of citation transparency, and accessibility on standard hardware [8]. This system addresses these gaps by implementing a local-first architecture that performs document processing and retrieval on-device, while maintaining factual grounding with explicit citations [2, 9].

2.2 Related Theory

The theoretical foundation of proposed system relies on several core concepts:

Transformer Architecture: The transformer model, introduced by Vaswani et al. [10], revolutionized natural language processing with self-attention mechanisms that enable efficient modeling of long-range dependencies in text. Transformers serve as the backbone for both embedding generation and language modeling in system. However, standard transformers are limited in the sequence lengths they can process efficiently [11]. Transformers serve as the backbone for both embedding generation and language modeling in system.

Sentence Embeddings: Sentence-BERT [12] extends BERT to produce semantically meaningful sentence embeddings using siamese and triplet networks. These embeddings allow meaningful comparison of text chunks and queries in a dense vector space, critical for effective semantic retrieval.

Dense Vector Retrieval: Dense retrieval algorithms such as DPR and indexing structures like HNSW [1] enable fast and scalable similarity search in high-dimensional spaces. HNSW graphs provide logarithmic query complexity and high recall, essential for real-time retrieval in large document collections.

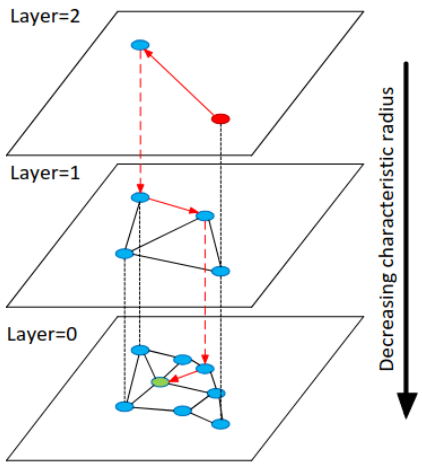

Figure 2.1: HNSW Visualization. Reproduced from Malkov et al.[1]

Retrieval-Augmented Generation (RAG): RAG models integrate retrieved external knowledge into LLM input to ground generation outputs, mitigating hallucinations and improving answer reliability [2, 9]. This approach blends parametric and non-parametric memory, enhancing performance on tasks requiring up-to-date or domain-specific information.

Chunking: Chunking is the process of splitting large document into small digestible units known as chunks. The main goal of chunk is to segment the complex data into more simpler blocks which helps to improve the accuracy. The important of chunking is massive as LLM have strict context size limits and proper chunking ensures that the context is not exceeded saving tokens as well. In chunking we generally talk about two major terms: chunk size which refers to the word/token size of each chunk and chunk overlays which is basically the certain words/token that is present in two adjacent chunks.

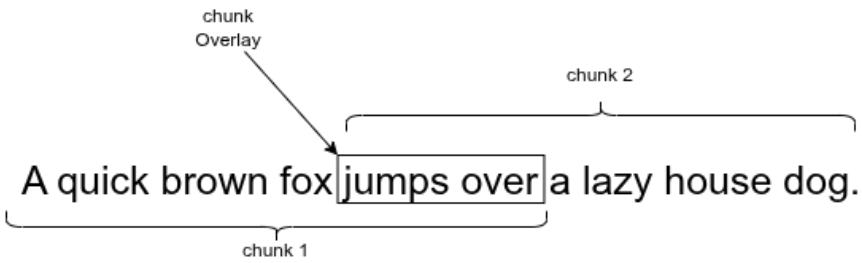

Figure 2.2: Chunking with chunk-size of 6 words and chunk-overlay of 2 words

Citation and Provenance: Transparent AI systems must provide citations to trace answers back to source documents, enabling verification and trust [6]. The system incorporates explicit citation metadata during chunking and answer generation to fulfill this need.

Local-First and Privacy-Preserving Design: Recent surveys emphasize the importance of privacy-preserving architectures for document processing, recommending local-first designs that minimize cloud dependencies to protect sensitive user data [8].

Together, these theories underpin system's design, enabling it to deliver accurate, transparent, and privacy-conscious document querying and educational content generation.

3. Methodology

3.1 Methodology

The methodology of the Intelligent Document Query System describes the systematic approach used to process PDF documents, generate embeddings, retrieve relevant information, answer user queries, and generate MCQs. The project follows a modular architecture to ensure privacy, efficiency, and accurate semantic understanding.

3.1.1 Retrieval-Augmented Generation (RAG) Architecture

RAG is a framework that combines *retrieval-based information fetching* with *generative language models* [2]. In this architecture, a user query is first used to retrieve relevant context from a knowledge base (the processed PDFs stored in ChromaDB), and the retrieved context is then passed to an LLM to generate accurate, context-aware answers.

This approach is effective because it allows the LLM to ground its responses in actual document content rather than relying solely on its pre-trained knowledge, reducing hallucinations and increasing factual accuracy [9]. RAG systems have been shown to outperform parametric-only models on knowledge-intensive tasks by combining parametric memory (the LLM's trained knowledge) with non-parametric memory (the external document collection) [2].

3.1.2 Document Processing and Chunking

Uploaded PDF documents are first converted into raw text using **pymupdf4llm** with page-level chunking enabled, returning a list of `(page_number, text)` tuples. The text is then divided into smaller segments called **chunks** using LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=384` tokens and `chunk_overlap=50` tokens (approximately 13%). This overlap was chosen to balance the trade-off between context completeness and embedding efficiency [7]. Too large a chunk can dilute semantic focus, while too small a chunk may lose context, reducing retrieval relevance.

While fixed-size chunking provides a reliable and reproducible baseline, it does not always respect the semantic boundaries of a document. A known limitation is the loss of cross-sentence context after splitting — for instance, a pronoun such as *"It"* in *"Berlin is the capital of Germany. It is known for its rich history"* loses its referent entirely when the two sentences fall into separate chunks. Late chunking addressed this by generating embeddings over the full document context prior to splitting, preserving coreference and contextual meaning within each chunk [4].

Rather than embedding each chunk independently, the system implements **windowed late chunking**: the full document token sequence is processed in a single transformer forward pass so that each token's representation is informed by its surrounding document context before chunk-level mean pooling occurs. Because documents regularly exceed the 8,192-token model context limit, a sliding window is applied — the full token sequence is split into overlapping windows of 8,192 tokens with a stride of 7,168 tokens (window overlap = 1,024 tokens between adjacent windows). Each window is embedded independently and chunk embeddings that straddle a window boundary are assigned to whichever window first encountered them, following a keep-first-assignment deduplication policy. Each chunk is stored with metadata {`chunk_index, document_id, page_number, text`} to enable traceable citations. Hybrid frameworks that further integrate spatial and semantic analysis [13] were also explored as complementary strategies.

3.1.3 Embedding Generation

Each text chunk is converted into a vector embedding using **nomic-embed-text-v1.5** (`nomic-ai/nomic-embe` via the HuggingFace Transformers library), a transformer-based model that produces 768-dimensional dense embeddings and supports a maximum context window of 8,192 tokens [10]. The model is specifically designed for asymmetric retrieval and requires asymmetric task prefixes: corpus chunks are prefixed with `search_document:` during indexing, while queries are prefixed with `search_query:` at retrieval time. Omitting or mismatching these prefixes degrades retrieval quality significantly. All inference runs on CPU using PyTorch with the number of threads set to the available CPU core count.

These embeddings capture the semantic meaning of the text, ensuring that even if a user query uses different wording than the document, the system can still retrieve the most relevant chunks based on meaning rather than exact word matches [12].

3.1.4 Retrieval Algorithm

For semantic search, the system uses **ChromaDB** with **Hierarchical Navigable Small World (HNSW)** indexing [1] for approximate nearest neighbour retrieval. Each document's chunks, embeddings, and page numbers are stored in a ChromaDB collection keyed by the **SHA-256 hash** of the PDF file path, ensuring stable and collision-free collection naming across sessions. The `upsert()` operation is used for storage so that re-indexing the same document is safe and idempotent — existing entries are updated rather than duplicated. On subsequent loads, ChromaDB serves all artefacts from disk, eliminating re-embedding entirely; cache load times are under 1 second regardless of document size.

Dense vector search is preferred over sparse keyword search because it captures semantic sim-

ilarity and retrieves conceptually relevant chunks even when the exact words do not match [3]. Dense passage retrieval has been shown to outperform traditional sparse retrieval methods like BM25 by 9–19% in terms of top-k passage retrieval accuracy [3]. The top-k ($k = 5$) candidates are retrieved via HNSW search using cosine similarity, filtered by a similarity threshold (> 0.5), and concatenated with metadata to form the LLM input context.

3.1.5 Answer Generation Workflow

Once relevant chunks are retrieved from ChromaDB, they are combined with the user query to form a contextual input for the LLM. The context and question are passed to Groq's **llama-3.1-8b-instant** API. The LLM is instructed to answer strictly from the provided context and cite the source page at the end of every answer in the format `(Source Page: [N])` [2, 14]. This citation mechanism addresses the transparency and provenance challenges identified in retrieval-augmented generation systems [9], ensuring every answer is verifiable against the source document.

3.1.6 MCQ Generation Process

The multiple-choice question (MCQ) module reuses the same retrieval stage as the Q&A mode. Retrieved chunks are passed to **llama-3.1-8b-instant** via Groq with a structured prompt requesting a four-option multiple choice question, a marked correct answer, a difficulty label, an explanation of at least 20 words, and a source-page citation. Key sentences and concepts are identified and transformed into questions, with one correct answer and multiple distractors. The system evaluates user responses and provides immediate feedback, enabling self-assessment and educational use [7].

3.2 Feasibility Study

This feasibility study evaluates the viability of developing the proposed system, an intelligent document query and MCQ generation system leveraging Retrieval-Augmented Generation (RAG) techniques.

Technical Feasibility

The proposed system uses mature technologies: **nomic-embed-text-v1.5** for contextual windowed late chunking embeddings, ChromaDB with HNSW indexing for fast semantic search, and large language models accessed via the Groq API. Local processing of document extraction and embeddings ensures privacy and offline operation for retrieval, while cloud-based LLM inference via Groq provides high-quality answer generation. The modular pipeline supports scalability and maintenance, with robust support for PDF documents.

Operational Feasibility

Targeted at students and professionals, the desktop application built with PyQt5 offers user-friendly interaction. The local-first design protects sensitive data, with minimal network usage limited to LLM API calls. Planned pilot testing and expert reviews ensure usability and reliability in practical settings.

Economic Feasibility

The system leverages open-source tools and lightweight models runnable on standard hardware, minimizing costs. Free-tier LLM API access via Groq reduces cloud expenses. The value added by advanced document querying and automated MCQ generation justifies the development investment.

Schedule Feasibility

A clear timeline from *Poush* to *Chaitra* divided the project into design, implementation, testing, and evaluation phases. The use of proven technologies and modular design promotes timely delivery with room for iterative improvements.

Overall, the system is technically sound, operationally practical, economically viable, and achievable within the planned schedule, confirming its feasibility.

3.3 Requirements Analysis

3.3.1 Functional Requirements

- **Document Processing:** Extract text from PDFs page-wise with metadata preservation (document name, page number, headers and subheaders).
- **Text Chunking:** Split text into 384-token chunks with 50-token overlap using `RecursiveCharacterTe` store metadata {`chunk_index, document_id, page_number, text`}.
- **Embedding Generation:** Locally generate contextual embeddings using **nomic-embed-text-v1.5** with asymmetric task prefixes (`search_document:` / `search_query:`).
- **Vector Storage:** Use ChromaDB + HNSW indexed by SHA-256 hash of file path; support incremental upsert updates.
- **Query Processing:** Encode queries with `search_query:` prefix; retrieve and filter top-5 chunks by cosine similarity (threshold > 0.5).
- **Answer Generation:** Generate fact-based answers with explicit source-page citations using Groq llama-3.1-8b-instant.
- **MCQ Generation:** Create four-option MCQs with marked answer, difficulty label, explanation ($\geq$20 words), and citation; provide immediate feedback.
- **User Interface:** PyQt5 desktop app for uploading PDFs, querying, and viewing results.
- **Logging:** Record queries, retrievals, answers, timestamps, and resource usage.

3.3.2 Non-Functional Requirements

- **Performance:** Sub-second retrieval; scale to tens of thousands of chunks.
- **Accuracy:** High retrieval precision; accurate, cited answers.
- **Privacy & Security:** Local processing for extraction and retrieval; minimal cloud data; no external document storage.
- **Usability:** Support diverse documents and question types; clear citations.
- **Robustness:** Handle corrupted, large, or non-English documents; stable operation.

3.3.3 Operational Requirements

- **Hardware:** Standard desktop without GPU; cloud only for LLM inference calls.
- **Software:** PyQt5 UI; pymupdf4llm for text extraction; nomic-embed-text-v1.5 via HuggingFace Transformers; ChromaDB.
- **Network:** Internet required for Groq LLM API; offline indexing and retrieval supported.
- **User:** Students and professionals; feedback mechanism for iterative improvement.

4. Experimental Setup

4.1 Evaluation Framework

To evaluate the performance and effectiveness of the system, a comprehensive assessment was conducted based on diverse document types, carefully designed test queries, and multiple quantitative and qualitative metrics.

Datasets:

The evaluation corpus comprises 31 documents drawn from diverse academic and technical domains including machine learning, computer vision, NLP, systems security, physics, and biology. Documents are stratified into four tiers by length: short (6 documents, 8–13 pages), medium (12 documents, 12–26 pages), large (12 documents, 21–75 pages), and very large (1 document, 165 pages). Total corpus size is 904 pages and approximately 1 million tokens. This diversity ensures the system's robustness across different content structures and domains [2, 8].

Test Query Sets:

A ground-truth evaluation set of 171 questions was constructed manually across 28 documents. Each entry contains the question, ground-truth answer, the correct source page number, and 2–5 distinctive keywords expected to appear in the relevant chunk. Question counts are proportional to document length: 2 questions for short documents, 4–5 for medium, and 10 for large documents. Questions span multiple difficulty levels including fact-based queries, inferential questions, and document-specific content retrieval [10, 3].

Evaluation Metrics:

- *Retrieval Metrics:* Recall@5, Precision@5, and Mean Reciprocal Rank (MRR) were used to measure how effectively the system retrieves relevant document chunks [1, 15].

$$\text{Precision@k} = \frac{\text{\# relevant results in top k}}{k}$$

$$\text{Recall@k} = \frac{\text{\# relevant results in top } k}{\text{total \# relevant results}}$$

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}$$

  Relevance is determined by keyword matching: a chunk is considered relevant if it contains 2–5 distinctive ground-truth keywords associated with the query.

- *MCQ Structural Metrics:* Five automatic structural metrics are computed for MCQ evaluation [6]:
  - Format validity: 4 options + marked answer + explanation present
  - Answer extractability: Correct answer letter (A–D) parseable from response
  - Explanation completeness: Explanation field contains $\geq$ 20 words
  - Citation presence: Response contains `(Source Page: [N])` citation
  - No option overlap: No two options share >75% Jaccard word similarity
- *Generation Metrics:* Answer correctness, factual accuracy, and citation accuracy (including page numbers and text excerpts) were evaluated to assess the quality and trustworthiness of generated answers [6, 9].
- *System Metrics:* Response time and memory usage were monitored to ensure the system performs efficiently on standard computing hardware [14, 16].

4.2 Performance Benchmarking

Baseline Comparisons:

The system was benchmarked against conventional keyword-based search (e.g., Ctrl+F) and a baseline large language model (LLM) that answers queries without retrieval augmentation. This comparison highlighted improvements in semantic understanding and factual grounding offered by Retrieval-Augmented Generation (RAG) methods [2, 7].

Test Scenarios:

- Single-document queries to evaluate focused retrieval and answer generation [10].
- Multi-document queries to assess the system's ability to integrate information from multiple sources [3].
- Ambiguous or vague questions to test robustness and system handling of uncertain queries [8].

Quality Assessment:

Domain experts performed a manual review of a sample of generated answers and MCQs to qualitatively judge correctness, relevance, and appropriateness [6, 1].

4.3 Experimental Procedure

The following step-by-step procedure was used for experimentation:

1. Uploaded the curated document collection (31 documents, 904 pages) into the system [2].
2. Executed the predefined query set of 171 questions, recording retrieval results and generated answers for each query [1].
3. Measured system response time and memory consumption during testing [14].
4. Conducted manual evaluation of answer quality, citation correctness, and MCQ relevance by domain experts [6].
5. Compared results against baseline methods to quantify improvements [3].

Data Collection Tools:
A logging system was integrated to capture queries, retrieved chunks, generated answers, timestamps, and resource usage metrics for detailed analysis and reproducibility [3].

4.4 Validation Methods

User Testing:
Tests were conducted by ourselves while development and post development of the application with various types and document and gathering feedbacks and improvement areas with measurement of effectiveness and practicality of the application.

Edge Case Testing:
System behaviour was also evaluated on challenging inputs, including corrupted or poorly scanned PDFs, very large documents exceeding typical token limits, and non-English text, to verify robustness and identify limitations [16].

It should be noted that given the limited project timeline, the evaluation is modest in scale. The sample sizes for both test queries (171 questions across 28 documents) and self testing are acknowledged as a limitation of this project's evaluation. Should the timeline permit, the scope of validation may be expanded to include a larger query set and a broader user study.

5. System Design

The system implements a Retrieval-Augmented Generation (RAG) architecture [2, 9] that combines dense passage retrieval with large language model generation. The system is designed as a modular desktop application where document processing, embedding generation, and retrieval operations occur locally, while answer generation leverages cloud-based LLM inference via the Groq API.

5.1 System Architecture

The system follows a pipeline architecture consisting of two primary workflows: the **Indexing Pipeline** (offline document processing) and the **Query Pipeline** (real-time question answering).

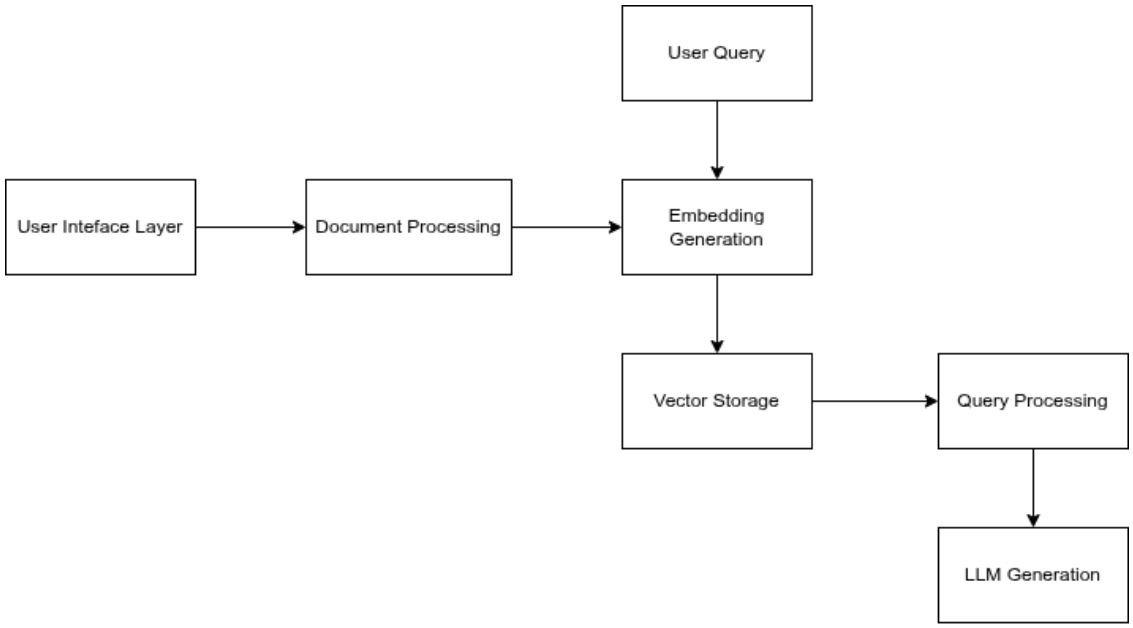

Figure 5.1: System Architecture

The architecture comprises six major components:

1. User Interface Layer built with PyQt5
2. Document Processing Module for text extraction and preprocessing
3. Embedding Generation Module for semantic vector encoding
4. Vector Storage Module with ChromaDB and HNSW indexing
5. Query Processing Module for retrieval and ranking
6. LLM Generation Module for context-aware answer generation

5.2 Document Processing Module

The document processing module performs text extraction using **pymupdf4llm** with `page_chunks=True`, returning a list of `(page_number, text)` tuples. Text is extracted page-wise with metadata preservation (document name, page number, headers and subheaders) to support accurate citation in generated responses.

Text Chunking Strategy

Following established RAG best practices [7], documents are split using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 384 tokens and a 50-token overlap (approximately 13%). The splitting hierarchy prioritizes paragraphs, sentences, and characters. Each chunk is stored with metadata {`chunk_index, document_id, page_number, text`} to enable traceable citations.

The windowed late chunking mechanism described in the Methodology chapter is then applied: rather than embedding each chunk independently, the full document token sequence is processed in a single transformer forward pass before chunk-level mean pooling occurs. For documents exceeding the 8,192-token model limit, a sliding window of 8,192 tokens with a stride of 7,168 tokens is applied [11], with a keep-first-assignment deduplication policy for chunks straddling window boundaries. This approach preserves coreference and contextual meaning within each chunk [4], with hybrid spatial-semantic segmentation [13] explored as a complementary strategy.

5.3 Embedding and Vector Storage Module

5.3.1 Embedding Model

The system uses **nomic-embed-text-v1.5** (`nomic-ai/nomic-embed-text-v1.5` via the HuggingFace Transformers library) [17], which produces **768-dimensional** dense embeddings and

supports a maximum context window of **8,192 tokens**. The model is designed for asymmetric retrieval and requires asymmetric task prefixes: corpus chunks are prefixed with `search_document:` during indexing, while queries are prefixed with `search_query:` at retrieval time. Omitting or mismatching these prefixes degrades retrieval quality significantly. All inference runs on CPU using PyTorch, following the dense passage retrieval paradigm [3].

5.3.2 Vector Database and Indexing

Dense embeddings are stored in **ChromaDB** (`persist_dir=./chroma_db/`) using **HNSW** indexing [1]. Each document's collection is keyed by the **SHA-256 hash** of the PDF file path, ensuring stable and collision-free collection naming across sessions. The `upsert()` operation is used for storage so that re-indexing the same document is safe and idempotent. On subsequent loads, ChromaDB serves all artefacts from disk, eliminating re-embedding entirely; cache load times are under 1 second regardless of document size. ChromaDB is selected for its native metadata filtering and suitability for datasets at the scale of thousands to tens of thousands of chunks, where it offers comparable performance to FAISS [15] with improved developer usability.

5.3.3 Query Processing and Retrieval Module

The retrieval module employs dense retrieval [3] to identify semantically relevant chunks. User queries are encoded using nomic-embed-text-v1.5 with the `search_query:` prefix, and the top-k ($k = 5$) candidates are retrieved via HNSW search using cosine similarity. Retrieved chunks are filtered using a similarity threshold (> 0.5) and concatenated with metadata to form the LLM input context. Dense retrieval is preferred over sparse methods (BM25, TF-IDF) due to improved handling of semantic variation and empirically demonstrated retrieval gains of 9–19% [3].

5.4 LLM Generation Module

Answer generation follows the retrieval-augmented generation paradigm [2, 9].

5.4.1 LLM and Prompting

The system uses the **Groq API** with **llama-3.1-8b-instant** (`max_tokens=1000`, default temperature) for efficient inference and cost-effective deployment. Retrieved chunks and metadata are formatted into structured prompts that constrain generation to the provided context. For Q&A mode the prompt instructs the model to answer strictly from the provided context and cite the source page as `(Source Page: [N])`. For MCQ mode the prompt requests a four-option question with marked answer, difficulty label, explanation ($\geq$20 words), and citation. Citations are rendered in the format `[Document: filename.pdf, Page: X]`, reducing hallucinations and ensuring verifiable outputs [9].

5.5 Class Diagram

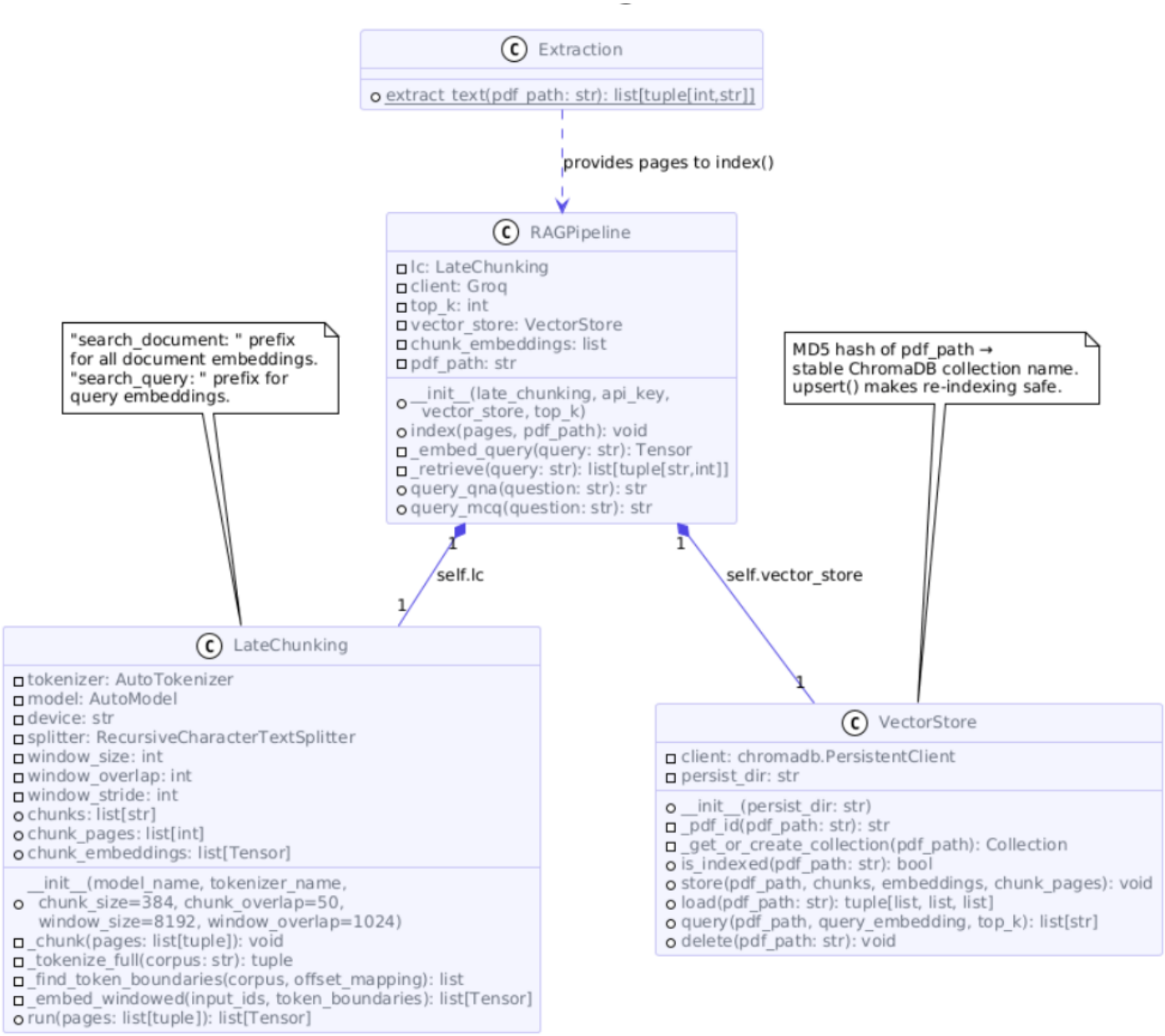

Figure 5.2: Class Diagram

5.6 Component Diagram

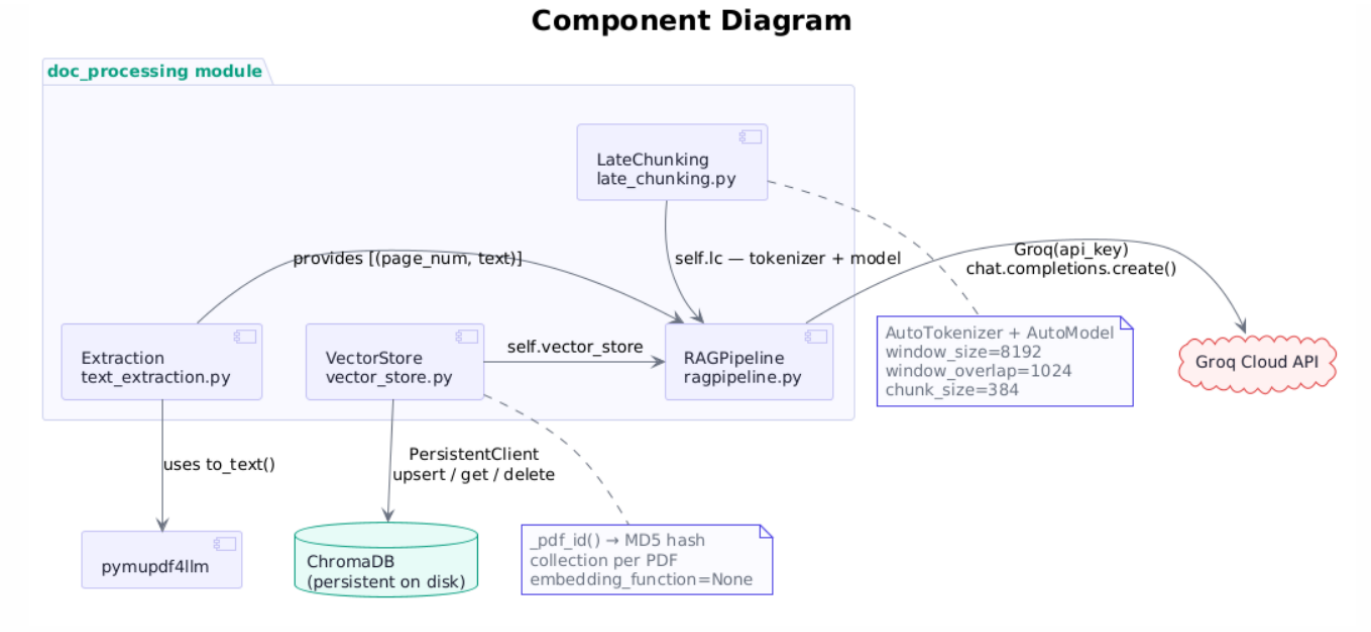

Figure 5.3: Component Diagram

5.7 Use Case Diagram

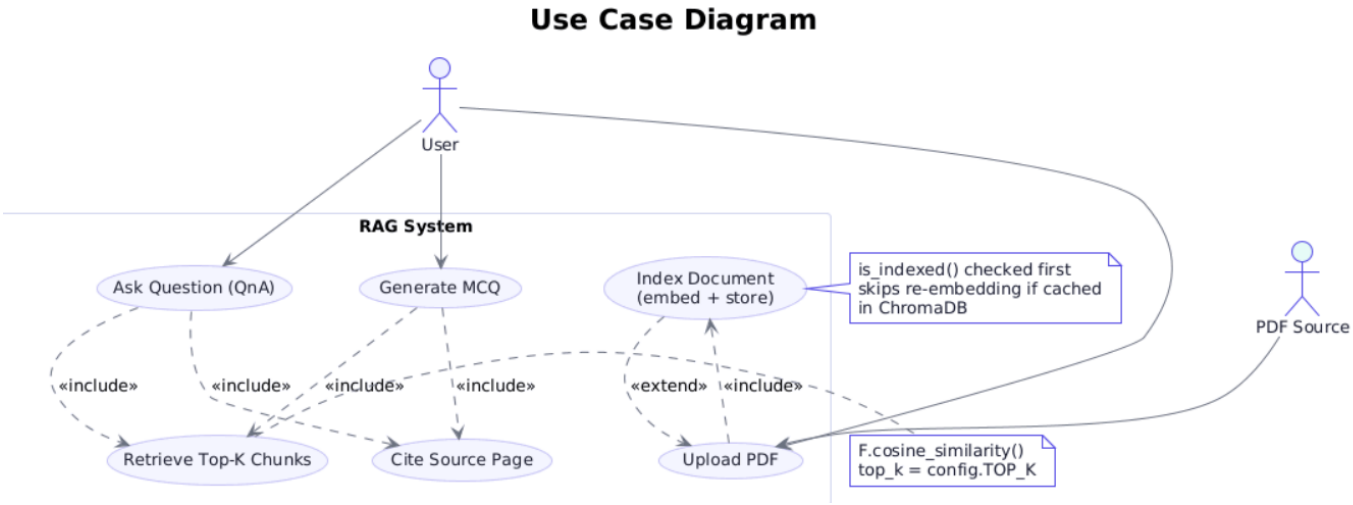

Figure 5.4: Use Case Diagram

5.8 Sequence Diagram

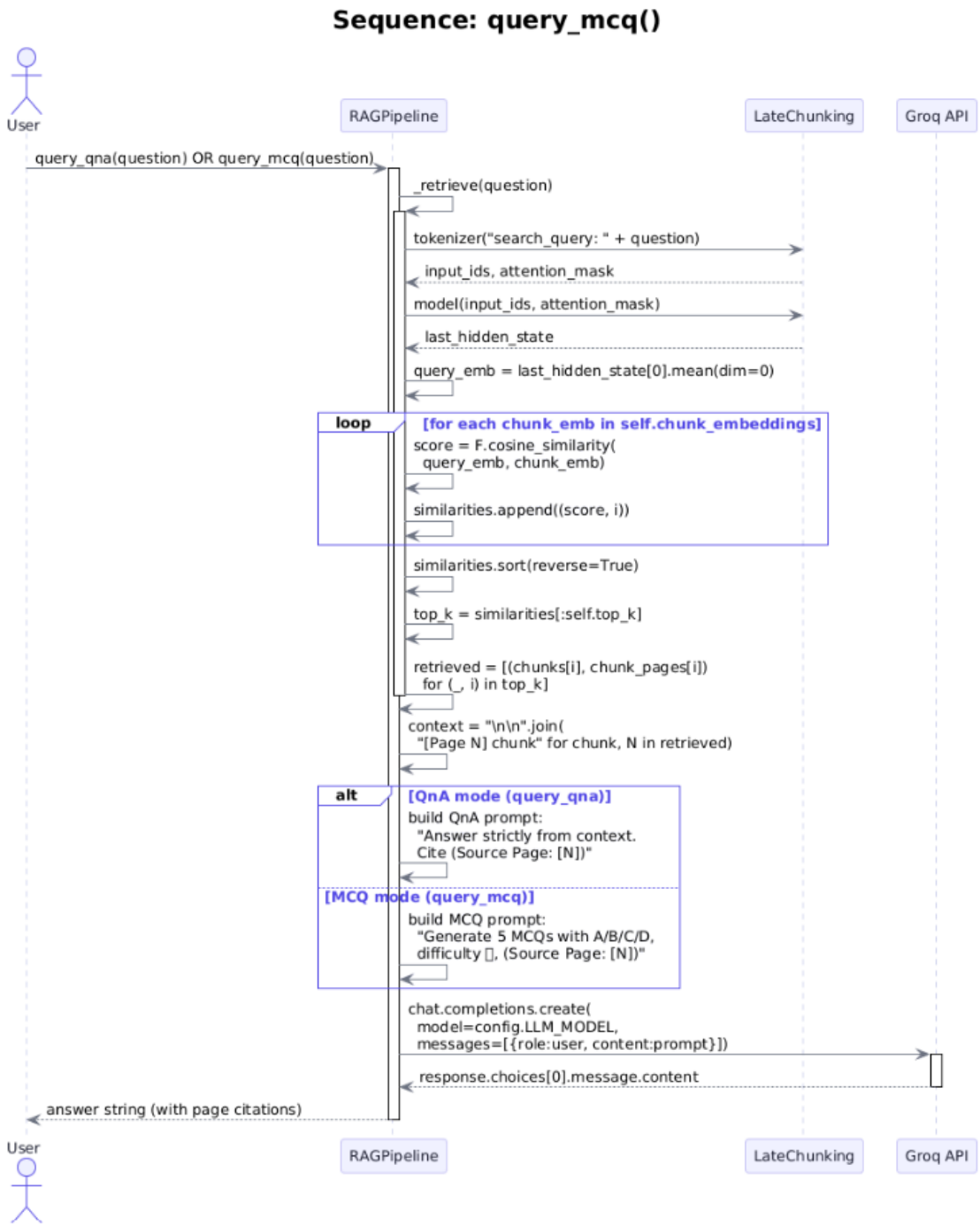

Figure 5.5: Sequence Diagram for query_mcq()

6. Result And Discussion

This chapter presents the complete evaluation of DocuMind system, covering indexing performance, retrieval quality and MCQ generation results. The evaluation was conducted on corpus of 31 documents of varying pages, chunks and token length with in total of about 904 pages and about a million tokens using a ground truth set of 171 manually curated questions spanning 28 documents. All experiments were performed on CPU only hardware without GPU acceleration.

6.1 System Interface

6.1.1 Question Answering Mode

The graphical user interface provides a split panel layout where loaded PDF is displayed on the left and AI assistant panel appears on the right. Users type natural language questions into input field; the system retrieves the top-5 relevant chunks and returns a grounded answer with explicit source page citation in format (source Page: [N]) also out of context answers are flagged as well.

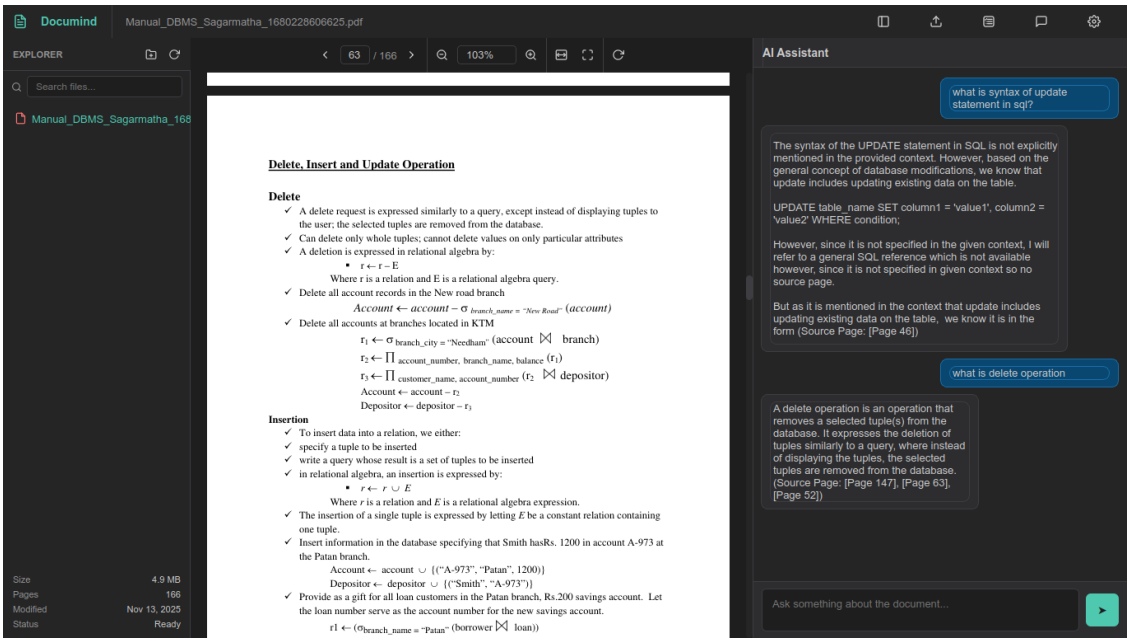

Figure 6.1: GUI Output in Question Answering Mode

6.1.2 Multiple Choice Questions Generation Mode

In MCQ mode the same retrieval pipeline supplies context chunks to LLM, which then generates a four option question in interactive mode and feature to reveal the answers. The MCQ generator also provides the correct answer, difficulty label, explanation and source page citation. The interface renders question with color-coded feedback on answer selection.

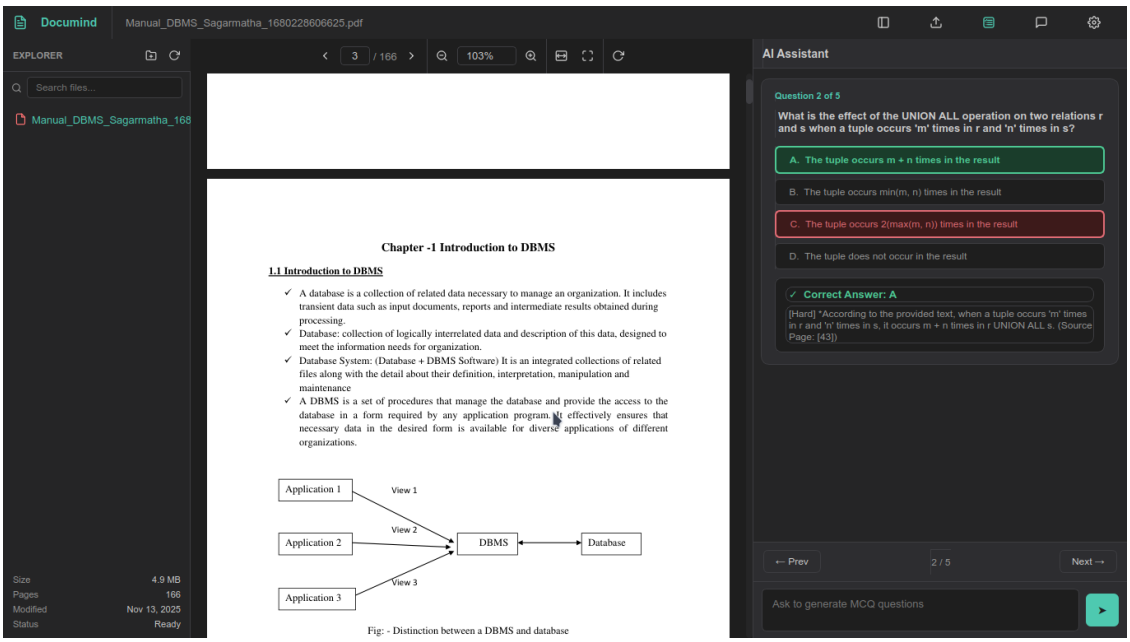

Figure 6.2: GUI Output in MCQ Mode

6.2 Tesing and Evaluation

6.2.1 Indexing Performance

Indexing covers raw PDFs into ChromaDB cached embeddings through five sequential stages: text extraction, chunking, tokenization, embedding and storage. All timing reflect fresh first-time indexing on CPU. Once a document is indexed subsequent loads from ChromaDB cache completes in under 1 second regardless of document size, eliminating re-embedding entirely.

Tier	Docs	Avg Pages	Avg Tokens	Avg Window	Avg Index Time
Short	*6*	*10.0*	*10,064*	*1.8*	*72 s*
Medium	*12*	*17.5*	*21,745*	*3.4*	*130 s*
Large	*12*	*39.1*	*55,166*	*8.2*	*327 s*
Very Large	*1*	*165.0*	*56,617*	*8.0*	*334 s*

Table 6.1: Indexing statistics by document tier

Embedding accounts for 94% of total indexing time. Text extraction contributes 5.7% while chunking, tokenization and storage together accounts for under 0.5%. Token count explains over 95% of variance in embedding time ($R^2 > 0.95$), confirming that indexing cost scales linearly with document length. The staircase pattern visible in Figure 6.3 directly reflects discrete window count increments: each step corresponds to one additional 8192 token sliding window.

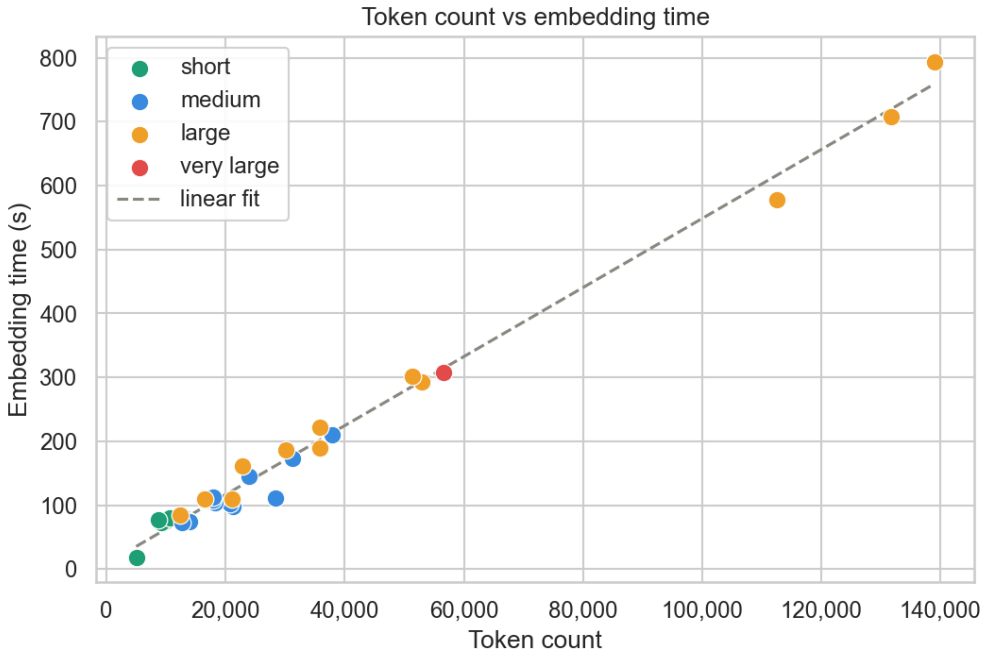

Figure 6.3: Token count vs embedding time Linear fit $R^2 > 0.95$

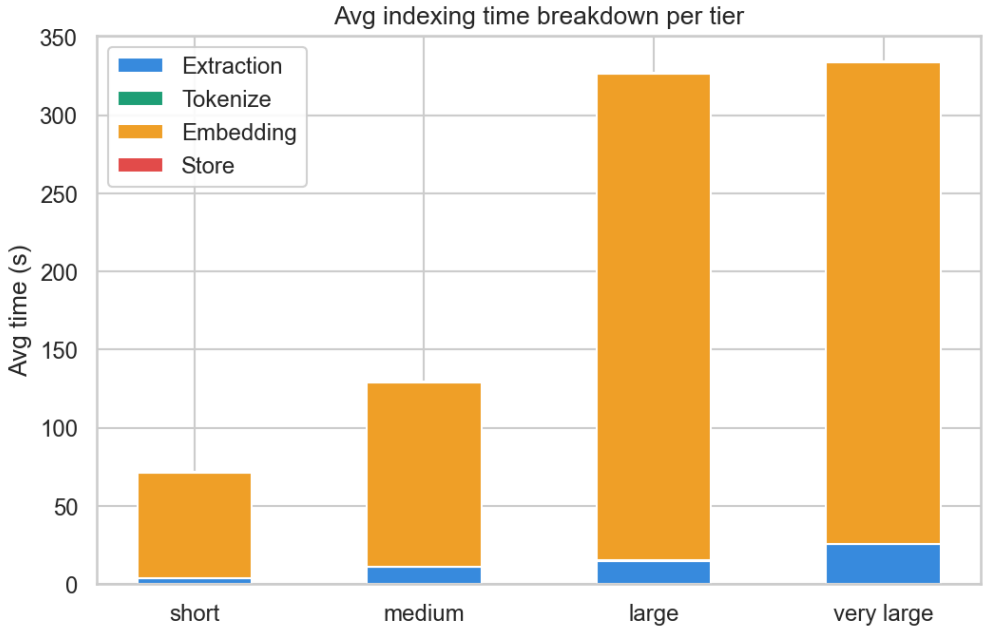

Figure 6.4: Average Indexing time breakdown per tier. Embedding dominates at 94% of total time

6.2.2 Retrieval Performance

Aggregrate Metrics

Table 6.2 repors aggregrate retrieval metrics across all 171 evaluation queries. All queries complete in under 600 ms on CPU, with a mean latency of 0.145 seconds. Retrieval requires only a single forward pass over the short query sequence (approximately 20 tokens) followed by exhaustive cosine similarity search — no re-embedding of document content occurs at query time.

Metric	Value	Notes
Recall@5	*71.3%*	*Relevant chunk in top-5 retrieved*
Precision@5	*32.0%*	*Mean fraction of top-5 that are relevant*
MRR	*0.547*	*Mean reciprocal rank of first relevant chunk*
Avg retrieval time	*0.145 s*	*CPU-only, exhaustive cosine search*
Avg cosine similarity	*0.524*	*Mean across all queries*
Genuine miss rate	*28.7%*	*Relevant content not retrieved at all*
Off-by-one page rate	*20.5%*	*Content retrieved but page number off by ±1*
Correct page rate	*55.6%*	*Strict page-level match*

Table 6.2: Aggregate retrieval metrics (n=171 queries)

An important distinction exists between content-level recall (71.3%) and correct-page rate (55.6%). Of the 76 page-level misses, 35 (20.5%) achieved recall = 1 i.e. the correct content was retrieved but attributed to an adjacent page due to chunk boundary placement at page transitions. Separating these off-by-one attribution errors from genuine retrieval failures reveals that the system's true content retrieval capability is meaningfully higher than the page-level figure alone suggests.

Per Document Analysis

Eight documents achieve perfect recall (100%), including several large documents, demonstrating that the system performs well on focused academic papers with distinctive terminology. The worst-performing document is Software-Defined Networking (30% recall) i.e. the largest document at 61 pages and 1,847 chunks — where the high chunk pool size introduces cosine similarity noise.

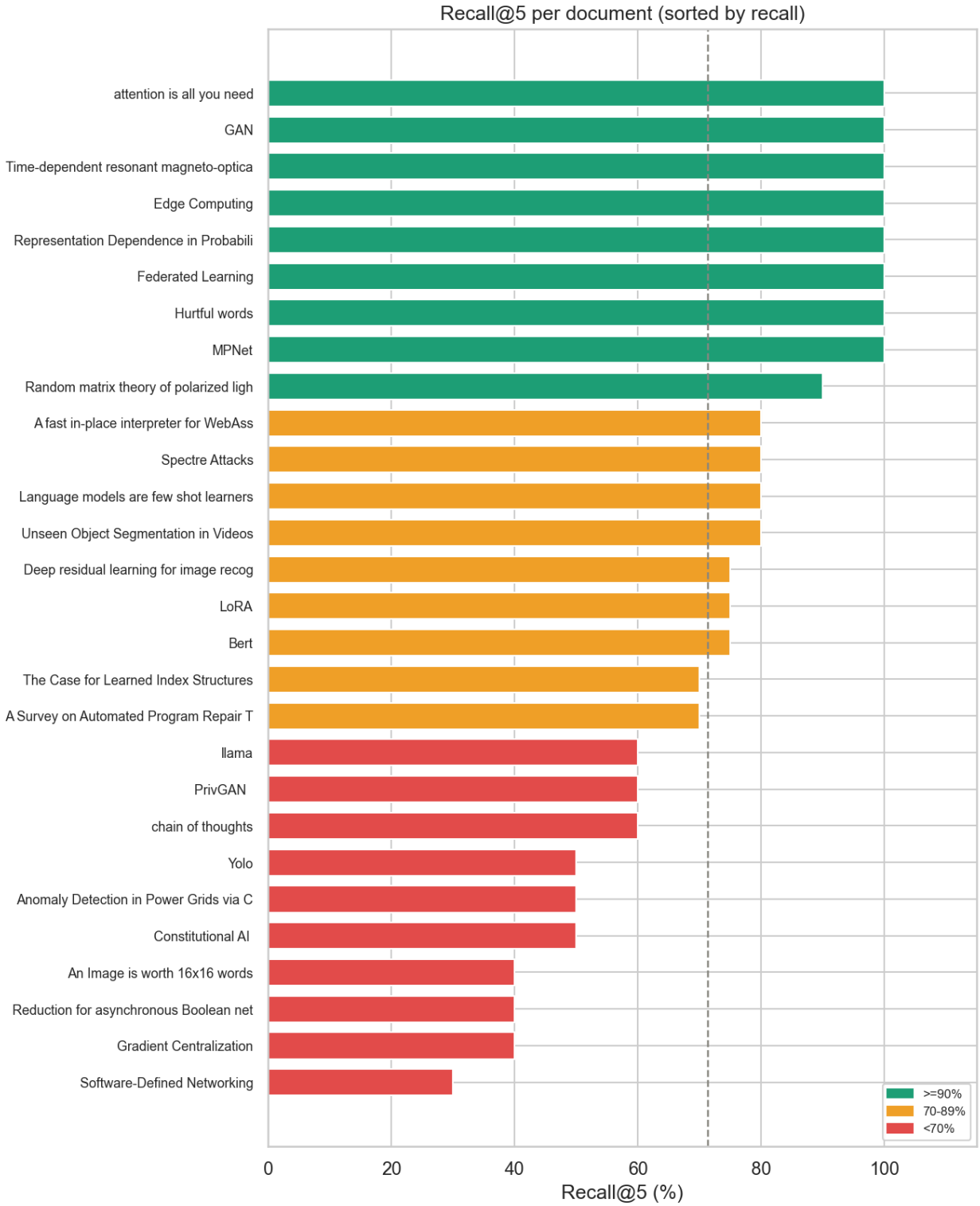

Figure 6.5: Recall@5 per document sorted best to worst. Dashed line is the overall mean (71.3%)

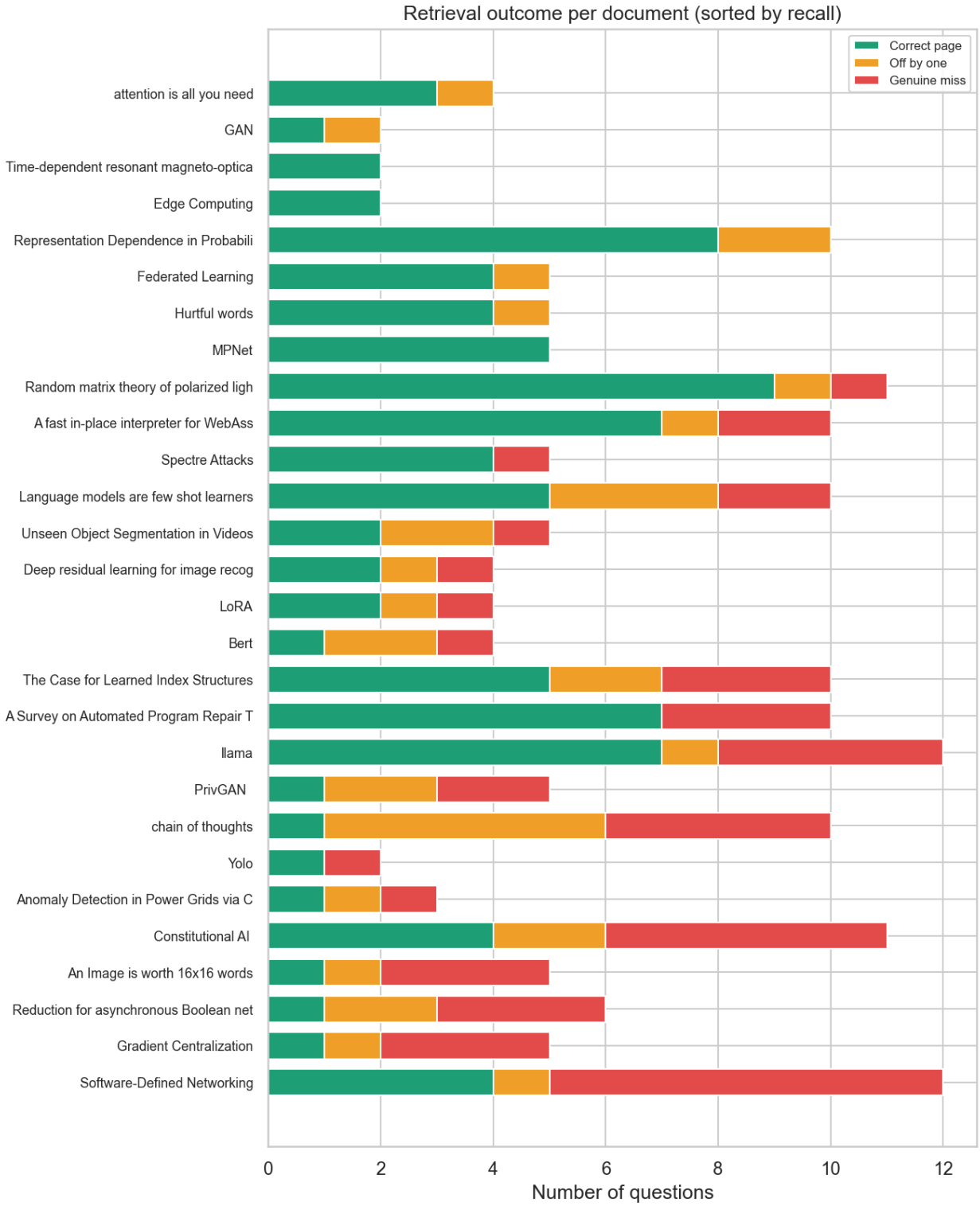

Figure 6.6: Retrieval outcome breakdown per document showing correct-page, off-by-one, and genuine miss counts

Effect of Document Length on Recall

A key finding is that raw document length is not the primary predictor of retrieval difficulty. Figure 6.7 shows recall reordered by page count: some large documents achieve perfect recall while shorter documents can fail. What matters more is chunk pool size and the specificity of domain terminology. This challenges the intuitive assumption that longer documents are inherently harder to retrieve from.

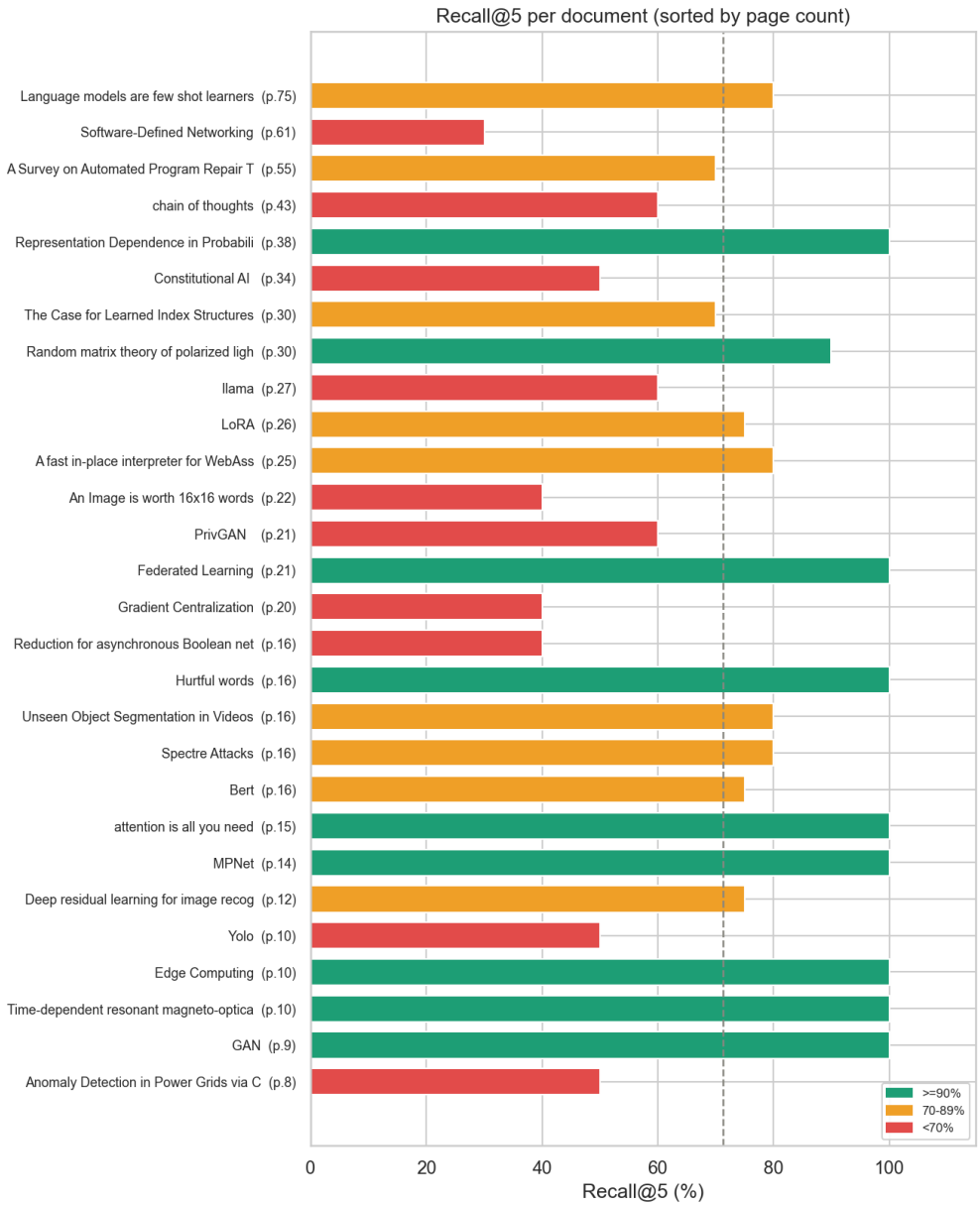

Figure 6.7: Recall reordered by page count

6.2.3 Retrieval Latency Distribution

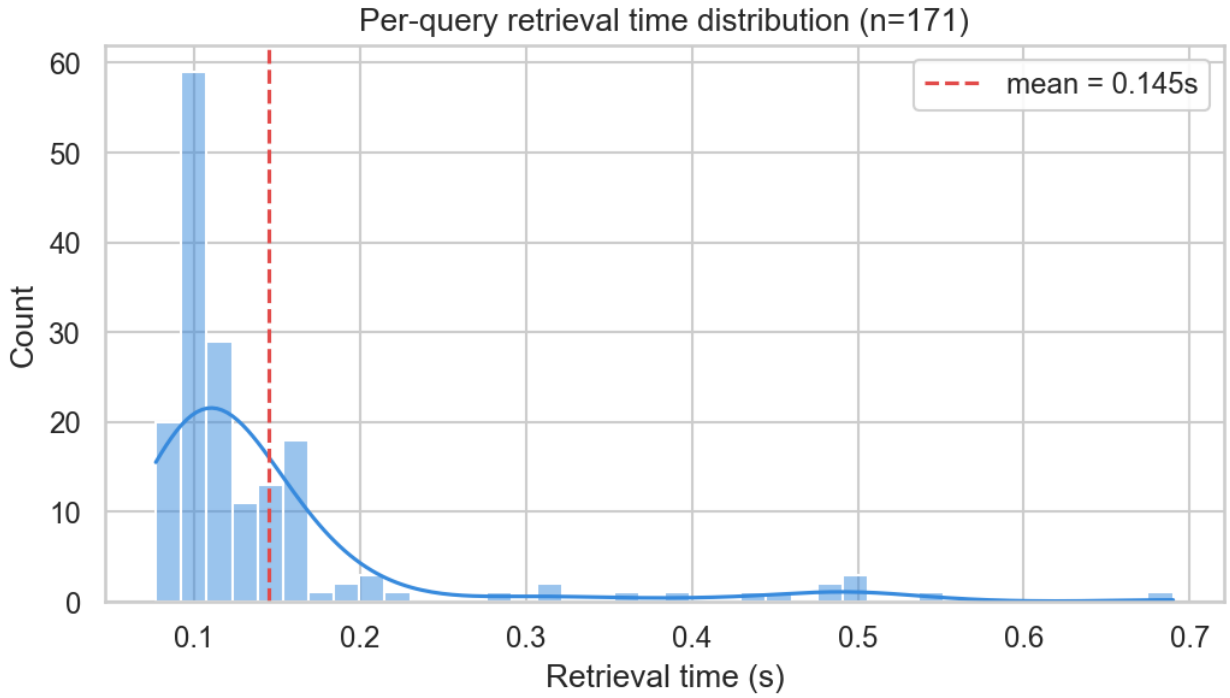

Figure 6.8: Per-query retrieval time distribution (n=171)

Failure Analysis

The 49 genuine misses are distributed across 20 of 28 documents. Three distinct failure patterns were identified.

Failure Pattern	Example Documents	Cause
Large chunk pools	*SDN (7 misses, 1,847 chunks), APR Survey (3 misses), GPT-3 (2 misses)*	*High chunk counts increase cosine similarity noise, making relevant chunks harder to rank in top-5*
Niche terminology	*Boolean Networks (3 misses), Random Matrix Theory (1 miss)*	*Specialised technical notation may not embed well in a general-purpose model*
Benchmark tables	*LLaMA (4 misses), ViT (3 misses)*	*Dense numerical tables with sparse prose produce low-distinctiveness embeddings*

Table 6.3: Failure pattern analysis

6.2.4 MCQ Generation Quality

The MCQ pipeline reuses the same retrieval stage as the Q&A mode. Retrieved chunks are passed to llama-3.1-8b-instant via Groq with a structured prompt requesting a four option question marked corrrect answer, difficulty label, explanation and source page citation. All 171 queries in evaluation run complete with zero API errors.

Metric	Result	Description
Format valid	*88.9%*	*4 options + marked answer + explanation present*
Answer extractable	*100.0%*	*A/B/C/D answer letter parseable from response*
Has explanation	*73.1%*	*Explanation field contains $\geq$ 20 words*
Citation present	*87.1%*	*Source page citation included in response*
No option overlap	*86.0%*	*No two options share ¿75% Jaccard word similarity*
Avg response time	*5.68 s*	*End-to-end including retrieval + LLM API call*
Errors	*0 / 171*	*Zero API failures across the complete run*

Table 6.4: MCQ automatic metric pass rates (n=171)

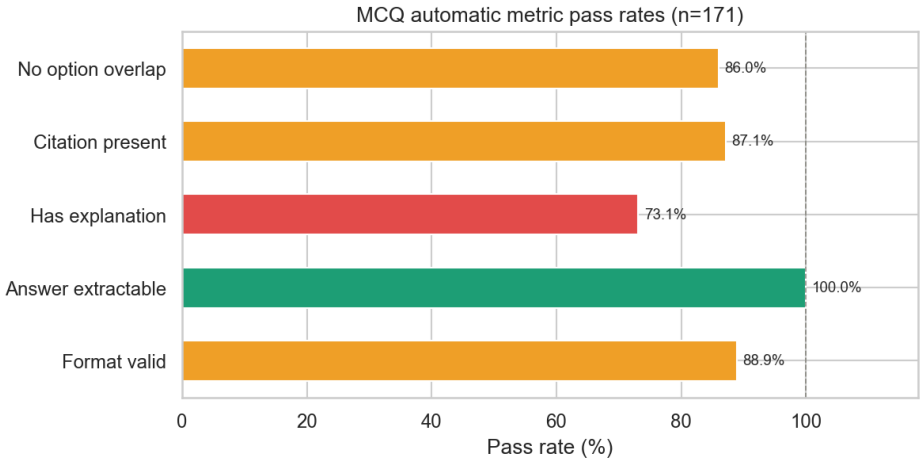

Figure 6.9: MCQ automatic metric pass rates

The standout result is 100% answer extractability — the model consistently produces a parseable A/B/C/D answer across all 171 questions, confirming that the structured generation prompt is robust. For a study-tool application this is the most critical property: students will always receive a clearly identified correct answer. Format validity at 88.9% reflects 19 questions where the response deviated from the expected structure. These correlate with slower response times (¿7 s), which occur when Groq rate limits reduce throughput mid-run. The average end-to-end response time of 5.68 seconds is dominated by the LLM API call; the retrieval step itself contributes only 0.145 seconds.

6.3 Discussion

Effectiveness of Windowed Late Chunking

The windowed late chunking approach addresses two fundamental limitations of standard RAG. First, standard independent chunk embedding discards document-level context: pronouns, acronyms, and implicit references cannot be resolved within an isolated chunk. Late chunking [4] resolves this by running the full-document transformer forward pass before mean-pooling individual chunk embeddings. Second, the 8,192-token context limit of nomic-embed-text-v1.5 [**?**] would otherwise prevent late chunking from being applied to documents larger than approximately 25 dense pages. A preliminary experiment applying late chunking with naive truncation at 8,192 tokens confirmed this limitation in practice — several documents in the evaluation corpus exceeded 10,000 tokens, meaning tail content was entirely discarded and retrieval quality on these longer documents was noticeably degraded. The sliding window extension removes this constraint at the cost of proportionally more computation, which the profiling data confirms is strictly linear with token count [11].

Retrieval Quality in Context

A Recall@5 of 71.3% on CPU-only hardware, without re-ranking or query expansion, represents a competitive baseline for this class of RAG system. The failure analysis establishes that chunk pool size is the primary difficulty predictor rather than document length. This has a practical design implication: for very large documents with thousands of chunks, strategies such as document-level filtering before chunk retrieval, or a two-stage coarse-to-fine retrieval approach, would likely improve recall without adding significant latency. An important negative finding is that cosine similarity score is a weak confidence signal. The evaluation found no strong correlation between the cosine score of the top-retrieved chunk and whether that chunk actually contained the correct answer. This means the system cannot reliably use cosine score as a downstream indicator of answer quality, and future work should explore re-ranking or calibration approaches.

MCQ Response Time

The 5.68-second end-to-end MCQ response time may appear slow at first glance. However, it is important to note that 0.145 seconds of this is local retrieval, and the remaining 5.5 seconds is Groq API network latency under rate limiting. This is an external dependency constraint, not a system bottleneck. Upgrading to a higher Groq API tier or switching to a locally-deployed LLM would bring response times down substantially.

7. Conclusion

This project successfully designed and implemented DocuMind, a localized Retrieval-Augmented Generation (RAG) system that bridges the gap between high-performance AI and data privacy. By integrating windowed late chunking with a CPU-optimized vector pipeline, the system effectively overcomes the three primary hurdles of general-purpose AI: lack of document-specific context, hallucination, and the privacy risks inherent in cloud-based processing.

The evaluation across a million-token corpus confirms that sophisticated semantic retrieval is achievable on standard consumer hardware. With a Recall@5 of 71.3% and a mean retrieval latency of under 150ms, DocuMind provides a responsive, desktop-class experience. Notably, the MCQ generation pipeline achieved 100% answer extractability, proving that grounding LLM responses in local retrieved context nearly eliminates the ”hallucination” issues common in broader AI tools.

A significant analytical contribution of this work is the finding that document length is not a primary predictor of retrieval difficulty. Instead, performance is dictated by terminological specificity and chunk density. Furthermore, error analysis revealed that 20.5% of retrieval misses were merely off-by-one page boundary artifacts, suggesting that the system’s ”true” semantic accuracy is substantially higher than strict metrics indicate.

DocuMind demonstrates that a privacy-conscious, resource-efficient research assistant is not only possible but practical for students and professionals. Future development will focus on GPU acceleration for faster initial indexing, OCR integration for scanned archives, and citation post-validation to further refine the reliability of automated educational content.

8. Limitation And Future Enhancements

8.1 Limitation

While the DocuMind system successfully demonstrated the capability to handle PDFs of various sizes and densities while generating grounded answers and MCQs, several limitations were observed during the development and testing phases:

- **Embedding Cost**: Indexing time scales linearly with token count, reaching approximately 327 s for a 39-page document and 334 s for a 165-page document on CPU. While this is a one-time cost eliminated on subsequent loads via ChromaDB caching, it makes first-time indexing slow for large document collections without GPU acceleration.
- **Retrieval Degradation on Large Documents**: The evaluation identified chunk pool size as the primary predictor of retrieval difficulty. The largest document (Software-Defined Networking, 1,847 chunks) achieved only 30% Recall@5 due to cosine similarity noise in a large pool, compared to the overall mean of 71.3%.
- **Off-by-One Page Attribution**: 20.5% of evaluation queries retrieved the correct content but attributed it to an adjacent page due to chunk boundary placement at page transitions. This affects citation accuracy even when content retrieval is correct.
- **Citation Reliability**: Although citations are correct in most cases, some responses have mismatched citations and in rare cases the LLM hallucinates the source page number entirely.
- **Initialization Dependency**: The **nomic-ai/nomic-embed-text-v1.5** model requires download from HuggingFace on first use, adding initialisation time and requiring an internet connection for the first run.
- **Image and Scanned PDF Support**: The system uses pymupdf4llm for text extraction, which handles digitally-born PDFs cleanly but does not support scanned documents or image-embedded equations. PDFs with no extractable text are not supported at all.
- **Session Not Persistent**: In the current implementation, opened PDFs, chat history, and generated MCQs do not persist between application sessions.
- **No Score Record**: The system can mark MCQ answers as correct or incorrect but does not maintain a cumulative record of total correct or incorrect answers across a session.

8.2 Future Enhancements

- **GPU Acceleration and Batch Embedding**: The embedding cost on CPU can be significantly reduced by adding optional GPU support via CUDA. Additionally, batch embedding multiple documents in parallel and pre-loading the model at startup rather than per-document would reduce perceived indexing time, making the system practical for institutional or large-scale deployments.

- **Improved Citation Grounding**: Citation reliability can be improved by adding a post-generation validation step that cross-checks the cited page number against the actual retrieved chunk metadata, and by incorporating a re-ranking stage that scores retrieved chunks by both semantic similarity and positional confidence.

- **OCR and Image-Embedded PDF Support**: Integration of an OCR engine such as Tesseract via pdf2image would enable processing of scanned documents and PDFs with image-embedded text and equations, significantly expanding the range of supported document types.

The following enhancements are straightforward to implement and are planned as immediate next steps:

- **Local Model Caching**: The nomic-embed-text-v1.5 model can be cached locally on first run using HuggingFace's `cache_dir` parameter, allowing fully offline operation after initial setup.

- **Persistent Sessions**: A local SQLite database can store chat history, opened documents, and generated MCQs between sessions, allowing users to resume previous work and build a personal question bank over time.

- **Score Tracking**: A score recording system that tracks correct and incorrect MCQ answers within and across sessions, displaying cumulative performance statistics at the end of each quiz.

- **UI Themes**: Support for multiple colour themes and an improved chat interface layout would enhance the visual experience and accessibility of the application.

References

[1] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs, August 2018. arXiv:1603.09320 [cs].

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, April 2021. arXiv:2005.11401 [cs].

[3] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense Passage Retrieval for Open-Domain Question Answering, September 2020. arXiv:2004.04906 [cs].

[4] Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao. Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models, July 2025. arXiv:2409.04701 [cs].

[5] Carlo Merola and Jaspinder Singh. Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation, April 2025. arXiv:2504.19754 [cs].

[6] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. Evaluation of Retrieval-Augmented Generation: A Survey. volume 2301, pages 102–120. 2025. arXiv:2405.07437 [cs].

[7] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. Retrieval-Augmented Generation for AI-Generated Content: A Survey, June 2024. arXiv:2402.19473 [cs].

[8] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions, October 2024. arXiv:2410.12837 [cs].

[9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models: A Survey, March 2024. arXiv:2312.10997 [cs].

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. arXiv:1706.03762 [cs].

[11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.

[12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, August 2019. arXiv:1908.10084 [cs].

[13] Prashant Verma. S2 Chunking: A Hybrid Framework for Document Segmentation Through Integrated Spatial and Semantic Analysis, January 2025. arXiv:2501.05485 [cs].

[14] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. A Survey on Retrieval-Augmented Text Generation, February 2022. arXiv:2202.01110 [cs].

[15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, February 2017. arXiv:1702.08734 [cs].

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, May 2019. arXiv:1810.04805 [cs].

[17] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder.

Appendix A: Complete Evaluation Corpus

Table 1 lists all 31 documents in the evaluation corpus with their tier, page count, token count, chunk count, window count, and total first-time CPU indexing time. This is the full data underlying the tier statistics reported in Table 6.1.

Table 1: Complete evaluation corpus (31 documents). Tokens counted by nomic-embed-text-v1.5 tokenizer. Win = number of 8,192-token sliding windows. Index time is first-time CPU indexing; cached reloads are under 1 s.

#	Document	Tier	Pgs	Tokens	Chunks	Win	Idx
1	Anomaly Detection in Power Grids via Context-Agnostic Learning	Short	8	9,232	155	2	76.8
2	EU AI ACT	Short	13	10,556	220	2	83.5
3	Edge Computing	Short	10	13,205	237	2	84.6
4	GAN	Short	9	8,631	123	2	80.7
5	Time-dependent Resonant Magneto-optical Rotation	Short	10	4,978	92	1	21.6
6	YOLO	Short	10	13,783	190	2	82.3
7	An Image is Worth 16x16 Words	Medium	22	28,343	343	4	119.6
8	BERT	Medium	16	21,259	284	3	103.9
9	Deep Residual Learning for Image Recognition	Medium	12	20,780	263	3	111.7
10	Gradient Centralization	Medium	20	18,170	223	3	110.0
11	Hurtful Words	Medium	16	31,214	452	5	183.9
12	LoRA	Medium	26	37,853	420	6	270.8
13	MPNet	Medium	14	17,179	211	3	113.0
14	PrivGAN	Medium	21	23,844	329	4	154.2
15	Reduction for Asynchronous Boolean Networks	Medium	16	17,932	219	3	113.4
16	Spectre Attacks	Medium	16	17,833	294	3	117.9
17	Unseen Object Segmentation in Videos	Medium	16	13,884	190	2	77.9
18	Attention is All You Need	Medium	15	12,656	180	2	77.8
19	A Survey on Automated Program Repair Techniques	Large	55	112,604	1,636	16	603.8

(continued on next page)

(continued from previous page)

#	Document	Tier	Pgs	Tokens	Chunks	Win	Idx
20	A Fast In-Place Interpreter for WebAssembly	Large	25	21,079	339	3	115.1
21	Constitutional AI	Large	34	52,952	607	8	304.4
22	Federated Learning	Large	21	16,429	290	3	115.6
23	Language Models are Few-Shot Learners (GPT-3)	Large	75	131,679	1,356	19	741.8
24	Neural Network Analysis of Neutron and X-ray Reflectivity	Large	30	12,412	199	2	89.4
25	Random Matrix Theory of Polarized Light Scattering	Large	30	22,825	371	4	170.2
26	Representation Dependence in Probabilistic Inference	Large	38	35,758	497	5	232.5
27	Software-Defined Networking	Large	61	139,028	1,847	20	844.6
28	The Case for Learned Index Structures	Large	30	30,073	436	5	194.5
29	Chain of Thoughts	Large	43	51,349	695	8	314.0
30	LLaMA	Large	27	35,812	419	5	201.5
31	Manual DBMS Sagarmatha	Very Large	165	56,617	793	8	334.3

Appendix B: Per-Document Retrieval Results

Table 2 presents complete per-document retrieval metrics across all 28 evaluated documents (171 queries total), sorted by Recall@5 from best to worst. Correct, Off-by-1, and Miss are absolute query counts.

Table 2: Per-document retrieval evaluation (28 documents, 171 queries). R@5 = Recall@5. P@5 = Precision@5. MRR = Mean Reciprocal Rank. Avg Sim = mean cosine similarity of top chunk. Off-by-1 = correct content on adjacent page. Miss = not retrieved.

Document	**Qs**	**R@5**	**P@5**	**MRR**	**Sim**	**Correct**	**OB1**	**Miss**	**Idx**
Edge Computing	2	1.000	0.600	1.000	0.522	2	0	0	0.284
Time-dep. Resonant Magneto-optical Rotation	2	1.000	0.200	0.291	0.464	2	0	0	0.228
GAN	2	1.000	0.200	0.666	0.465	1	1	0	0.164
Attention is All You Need	4	1.000	0.750	0.875	0.551	3	1	0	0.093
MPNet	5	1.000	0.720	0.900	0.531	5	0	0	0.091
Hurtful Words	5	1.000	0.600	1.000	0.545	4	1	0	0.112
Federated Learning	5	1.000	0.280	0.733	0.523	4	1	0	0.092
Representation Dependence in Prob. Inference	10	1.000	0.720	1.000	0.537	8	2	0	0.109
Random Matrix Theory of Polarized Light	10	0.900	0.440	0.578	0.500	9	1	1	0.109
Unseen Object Segmentation in Videos	5	0.800	0.360	0.700	0.481	2	2	1	0.091
Spectre Attacks	5	0.800	0.200	0.490	0.514	4	0	1	0.092
A Fast In-Place Interp. for WebAssembly	10	0.800	0.300	0.542	0.524	7	1	2	0.177
Language Models are Few-Shot Learners (GPT-3)	10	0.800	0.420	0.650	0.584	5	3	2	0.151
LoRA	4	0.750	0.300	0.625	0.524	2	1	1	0.108
Deep Residual Learning for Image Recognition	4	0.750	0.400	0.750	0.526	2	1	1	0.336
BERT	4	0.750	0.200	0.229	0.556	1	2	1	0.110

(continued on next page)

(continued from previous page)

Document	**Qs**	**R@5**	**P@5**	**MRR**	**Sim**	**Correct**	**OB1**	**Miss**	**Idx**
A Survey on Automated Program Repair	10	0.700	0.240	0.525	0.540	7	0	3	0.157
The Case for Learned Index Structures	10	0.700	0.240	0.445	0.527	5	2	3	0.108
PrivGAN	5	0.600	0.280	0.290	0.576	1	2	2	0.285
Chain of Thoughts	10	0.600	0.320	0.470	0.515	1	5	4	0.116
LLaMA	10	0.600	0.160	0.475	0.490	7	1	4	0.110
Anomaly Detection in Power Grids	2	0.500	0.100	0.500	0.422	1	1	1	0.084
YOLO	2	0.500	0.100	0.250	0.531	1	0	1	0.115
Constitutional AI	10	0.500	0.160	0.420	0.538	4	2	5	0.130
An Image is Worth 16x16 Words	5	0.400	0.160	0.200	0.539	1	1	3	0.210
Gradient Centralization	5	0.400	0.160	0.400	0.528	1	1	3	0.103
Reduction for Asynchronous Boolean Networks	5	0.400	0.120	0.267	0.477	1	2	3	0.107
Software-Defined Networking	10	0.300	0.140	0.233	0.512	4	1	7	0.294

Appendix C: Sample MCQ Generation Outputs

The following five examples show representative MCQ outputs generated by the system across different document domains. Each example was selected from queries where all five automatic quality metrics passed (format valid, answer extractable, has explanation, citation present, no option overlap).

C.1 Edge Computing — Easy

Source: Edge Computing.pdf Page: 3 Response time: 2.64 s

Question: By what amount was response time reduced when moving face recognition computation from cloud to the edge in the proof-of-concept platform?

A)	1–50 ms
B)	70–110 ms
C)	169 ms
D)	800–1000 ms

Correct Answer: C **Difficulty:** Easy

Explanation: According to the context, the response time is reduced from 900 ms to 169 ms when face recognition is moved from cloud to edge. (Source Page: [3])

C.2 Anomaly Detection in Power Grids — Medium

Source: Anomaly Detection in Power Grids.pdf Page: 1 Response time: 1.13 s

Question: What is the number of nodes in the simulated networks used in the numerical evaluation?

A)	Up to 100 nodes
B)	Up to 2,383 nodes
C)	Up to 1,000 nodes
D)	Up to 500 nodes

Correct Answer: B **Difficulty:** Medium

Explanation: The paper explicitly states that numerical simulations have been performed on networks with up to 2,383 nodes. (Source Page: [Page 1])

C.3 YOLO Object Detection — Easy

Source: Yolo.pdf Page: 6 Response time: 1.11 s

Question: What is the mAP score of the best Fast R-CNN model on the VOC 2007 test set?

A)	65.5%
B)	71.8%
C)	75.0%
D)	80.0%

Correct Answer: B **Difficulty:** Easy

Explanation: As mentioned in the context, the best Fast R-CNN model achieves a mAP of 71.8% on the VOC 2007 test set. (Source Page: [6])

C.4 LoRA — Easy

Source: LoRA.pdf Page: 7 Response time: 1.40 s

Question: When comparing LoRA with adapter baselines, what sequence length is used across all tasks?

A)	A sequence of 256
B)	A sequence of 128
C)	A batch size of 512
D)	A batch size of 128

Correct Answer: B **Difficulty:** Easy

Explanation: According to the text, to match the adapter baselines the authors use the same batch size for all tasks and a sequence length of 128. (Source Page: [Page 7])

C.5 Deep Residual Learning — Medium

Source: Deep_residual_learning.pdf Pages: 5–6 Response time: 0.94 s

Question: What competition did the ensemble of residual nets win, and what top-5 error rate did the 18-layer ResNet achieve on ImageNet?

A)	ImageNet competition; 28.07%
B)	ILSVRC competition; 9.33%
C)	ImageNet Large Scale Visual Recognition Challenge; 9.33%
D)	CIFAR-10 competition; 8.00%

Correct Answer: C **Difficulty:** Medium

Explanation: Table 3 refers to the competition as the ImageNet Large Scale Visual Recognition Challenge. The 18-layer ResNet top-5 error of 9.33% is listed in Table 2. (Source Page: [5][6])

Appendix D: Per-Document Indexing Time Breakdown

Table 3 presents the detailed indexing time breakdown for all 31 documents across four pipeline stages: text extraction, tokenization, embedding, and vector storage. All times are in seconds on CPU-only hardware. The embedding stage dominates at approximately 94% of total time, confirming the finding in Section 6.2.

Table 3: Per-document indexing time breakdown (seconds, CPU-only). Total = extraction + tokenization + embedding + storage.

Document	Tier	Pgs	Tokens	Chunks	Ext. (s)	Tok. (s)	Emb. (s)	Store. (s)
Anomaly Detection in Power Grids via Context-Agnostic Learning	Short	8	9,232	155	3.6	0.04	73.0	0.10
EU AI ACT	Short	13	10,556	220	3.2	0.05	80.1	0.16
Edge Computing	Short	10	13,205	237	4.8	0.06	79.5	0.15
GAN	Short	9	8,631	123	2.7	0.04	77.6	0.33
Time-dependent Resonant Magneto-optical Rotation	Short	10	4,978	92	2.8	0.02	18.6	0.14
YOLO	Short	10	13,783	190	5.6	0.06	76.0	0.63
An Image is Worth 16x16 Words	Medium	22	28,343	343	8.3	0.12	111.0	0.19
BERT	Medium	16	21,259	284	6.4	0.09	97.3	0.17
Deep Residual Learning for Image Recognition	Medium	12	20,780	263	9.5	0.08	101.5	0.60
Gradient Centralization	Medium	20	18,170	223	6.1	0.07	103.7	0.16
Hurtful Words	Medium	16	31,214	452	10.0	0.13	172.9	0.93
LoRA	Medium	26	37,853	420	60.0	0.15	210.4	0.23
MPNet	Medium	14	17,179	211	4.8	0.07	108.0	0.15
PrivGAN	Medium	21	23,844	329	8.7	0.10	145.2	0.19
Reduction for Asynchronous Boolean Networks	Medium	16	17,932	219	4.4	0.08	108.7	0.14
Spectre Attacks	Medium	16	17,833	294	5.5	0.08	112.2	0.17
Unseen Object Segmentation in Videos	Medium	16	13,884	190	3.9	0.06	73.8	0.14

(continued on next page)

(continued from previous page)

Document	Tier	Pgs	Tokens	Chunks	Ext. (s)	Tok. (s)	Emb. (s)	Store. (s)
Attention is All You Need	Medium	15	12,656	180	4.0	0.05	73.3	0.45
A Survey on Automated Program Repair Techniques	Large	55	112,604	1,636	24.4	0.47	578.0	0.86
A Fast In-Place Interpreter for WebAssembly	Large	25	21,079	339	5.3	0.09	109.5	0.22
Constitutional AI	Large	34	52,952	607	10.1	0.21	292.8	1.37
Federated Learning	Large	21	16,429	290	5.3	0.08	110.0	0.21
Language Models are Few-Shot Learners (GPT-3)	Large	75	131,679	1,356	32.1	0.50	708.3	0.84
Neural Network Analysis of Neutron and X-ray Reflectivity	Large	30	12,412	199	4.9	0.06	83.9	0.55
Random Matrix Theory of Polarized Light Scattering	Large	30	22,825	371	7.8	0.10	162.0	0.23
Representation Dependence in Probabilistic Inference	Large	38	35,758	497	9.9	0.17	221.9	0.48
Software-Defined Networking	Large	61	139,028	1,847	47.8	0.65	794.0	2.05
The Case for Learned Index Structures	Large	30	30,073	436	7.1	0.12	187.0	0.27
Chain of Thoughts	Large	43	51,349	695	12.3	0.25	301.0	0.50
LLaMA	Large	27	35,812	419	11.2	0.15	189.0	1.11
Manual DBMS Sagarmatha	Very Large	165	56,617	793	25.4	0.27	308.1	0.55

Appendix E: Windowed Late Chunking : Algorithm and Implementation

E.1 Algorithm

Algorithm 1 Windowed Late Chunking

Require:

pages — list of (*page_number*, *text*) tuples

chunk_size — 384 tokens

chunk_overlap — 50 tokens

window_size — 8192 tokens

window_stride — 7168 tokens (*window_size* − *window_overlap*)

Ensure:

chunk_embeddings — list of contextualised embedding vectors, one per chunk

corpus ← concatenate all page texts with `"\n\n"` separator

▷ Text-level chunking

chunks ← RECURSIVECHARACTERTEXTSPLITTER(*corpus*, *chunk_size*, *chunk_overlap*)

chunk_pages ← track source page number for each chunk

▷ Tokenize full document (no truncation)

prefixed_corpus ← `"search_document: "` + *corpus*

input_ids, *offset_mapping* ← TOKENIZER(*prefixed_corpus*)

T ← total token count

▷ Find token boundaries for each text chunk

for each chunk c_i in *chunks* **do**

  char_start, *char_end* ← character span of c_i in *corpus* ▷ Binary search over *offset_mapping*

  tok_start ← first token index where *token.start* $\geq$ *char_start*

  tok_end ← last token index where *token.end* $\leq$ *char_end*

  token_boundaries$[i]$ ← (*tok_start*, *tok_end*)

end for

▷ Sliding window embedding (keep-first assignment)

$chunk_embeddings[0 \,..\, N] \leftarrow$ **None**
$window_start \leftarrow 0$
while $window_start < T$ **do**
    $window_end \leftarrow \min(window_start + window_size,\ T)$ ▷ Single transformer forward pass over this window
    $hidden_states \leftarrow$ TRANSFORMERENCODER$\big(input_ids[window_start : window_end]\big)$
    for each chunk c_i with boundaries $(tok_start,\ tok_end)$ **do**
        if c_i already assigned **then continue** ▷ keep-first
        end if
        if c_i outside $[window_start,\ window_end)$ **then continue**
        end if
        $local_start \leftarrow \max(tok_start,\ window_start) - window_start$
        $local_end \leftarrow \min(tok_end,\ window_end) - window_start$ ▷ Mean-pool contextualised token vectors for this chunk
        $chunk_embeddings[i] \leftarrow$ MEAN$\big(hidden_states[local_start : local_end + 1]\big)$
    end for
    $window_start \leftarrow window_start + window_stride$
end while
▷ Fallback for any unassigned chunks
for each $chunk_embeddings[i]$ that is **None do**
    $chunk_embeddings[i] \leftarrow$ ZEROVECTOR$(hidden_size)$
end for
return $chunk_embeddings$

E.2 Implementation

```python
class LateChunking:
    """
    Windowed late chunking using nomic-embed-text-v1.5.

    Processes the full document in a single tokenization pass, then
    embeds overlapping windows of 8192 tokens with a stride of 7168.
    Each chunk is assigned to the first window that covers it
    (keep-first policy), and its embedding is the mean of the
    contextualised token hidden states within that window.
    """

    def __init__(
        self,
        model_name,
        tokenizer_name,
        chunk_size=384,
        chunk_overlap=50,
        window_size=8192,
        window_overlap=1024,
    ):
        self.tokenizer = AutoTokenizer.from_pretrained(
            tokenizer_name, trust_remote_code=True
        )
        self.model = AutoModel.from_pretrained(
            model_name, trust_remote_code=True
        ).to(config.DEVICE)
        self.model.eval()
        torch.set_num_threads(os.cpu_count())

        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
            separators=["\n\n", "\n", ". ", " ", ""],
        )
        self.window_size   = window_size
        self.window_stride = window_size - window_overlap

    def _chunk(self, pages):
        """Split page texts into overlapping chunks, tracking source page."""
        self.chunks, self.chunk_pages = [], []
        for page_num, text in pages:
```

```
            splits = self.splitter.split_text(text)
            self.chunks.extend(splits)
            self.chunk_pages.extend([page_num] * len(splits))

    def _tokenize_full(self, corpus):
        """
        Tokenize the full prefixed corpus without truncation.
        The 'search_document:' prefix is required by nomic-embed-text-v1.5
        for asymmetric retrieval -- omitting it degrades recall.
        """
        encoded = self.tokenizer(
            "search_document: " + corpus,
            return_tensors="pt",
            return_offsets_mapping=True,
            truncation=False,
            add_special_tokens=True,
        )
        return encoded["input_ids"], encoded["offset_mapping"]

    def _find_token_boundaries(self, corpus, offset_mapping):
        """
        Map each text chunk to a (tok_start, tok_end) token index range
        using binary search over the tokenizer's offset mapping.
        This avoids O(n^2) scanning for large documents.
        """
        prefix_len = len("search_document: ")
        offsets    = offset_mapping[0]
        tok_starts = [offsets[i][0].item() for i in range(len(offsets))]
        tok_ends   = [offsets[i][1].item() for i in range(len(offsets))]

        boundaries, current_pos = [], 0
        for chunk in self.chunks:
            start = corpus.find(chunk, current_pos)
            if start == -1:
                start = corpus.find(chunk)
            end = start + len(chunk)

            tok_s = bisect.bisect_left(tok_starts, start + prefix_len)
            tok_e = bisect.bisect_right(tok_ends,  end   + prefix_len) - 1
            # Skip padding/special tokens (offset = (0, 0))
            while tok_s < len(tok_starts) and tok_starts[tok_s] == 0:
                tok_s += 1
            boundaries.append((tok_s, max(tok_e, tok_s)))
```

```
            current_pos = end

        return boundaries

    def _embed_windowed(self, input_ids, token_boundaries):
        """
        Core sliding-window embedding with keep-first chunk assignment.

        Windows advance by window_stride tokens. Each chunk is embedded
        by mean-pooling its contextualised token vectors from the first
        window that fully or partially covers it.
        """
        T = input_ids.shape[1]
        chunk_embeddings = [None] * len(token_boundaries)

        # Ensure the last window always reaches the end of the sequence
        starts = list(range(0, max(T - self.window_size + 1, 1),
                            self.window_stride))
        if starts[-1] + self.window_size < T:
            starts.append(max(0, T - self.window_size))

        for win_start in starts:
            win_end = min(win_start + self.window_size, T)
            win_ids = input_ids[:, win_start:win_end].to(config.DEVICE)

            with torch.inference_mode():
                hidden = self.model(
                    input_ids=win_ids,
                    attention_mask=torch.ones_like(win_ids),
                ).last_hidden_state[0]          # [win_len, hidden_dim]

            for ci, (tok_s, tok_e) in enumerate(token_boundaries):
                if chunk_embeddings[ci] is not None:
                    continue                     # keep-first: already set
                if tok_e < win_start or tok_s >= win_end:
                    continue                     # chunk outside this window

                local_s = max(tok_s, win_start) - win_start
                local_e = min(tok_e, win_end)   - win_start
                if local_s > local_e:
                    continue

                chunk_embeddings[ci] = hidden[local_s:local_e + 1].mean(dim=0)
```

```python

        # Zero-vector fallback for any chunks that were missed
        hidden_size = self.model.config.hidden_size
        for ci in range(len(chunk_embeddings)):
            if chunk_embeddings[ci] is None:
                chunk_embeddings[ci] = torch.zeros(
                    hidden_size, device=config.DEVICE
                )

        return chunk_embeddings

    def run(self, pages):
        """End-to-end pipeline: pages -> contextualised chunk embeddings."""
        corpus = "\n\n".join(text for _, text in pages)
        self._chunk(pages)
        input_ids, offset_mapping = self._tokenize_full(corpus)
        token_boundaries = self._find_token_boundaries(corpus, offset_mapping)
        self.chunk_embeddings = self._embed_windowed(input_ids,
            token_boundaries)
        return self.chunk_embeddings
```

Listing 1: Windowed Late Chunking — `LateChunking` class